DeepMimic: Example-Guided Deep Reinforcement Learning of Physics-Based Character Skills

XUE BIN PENG, University of California, Berkeley
PIETER ABBEEL, University of California, Berkeley
SERGEY LEVINE, University of California, Berkeley
MICHIEL VAN DE PANNE, University of British Columbia

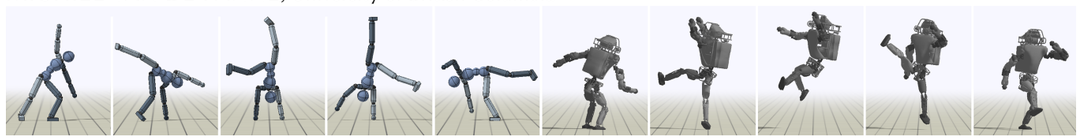


Fig. 1. Highly dynamic skills learned by imitating reference motion capture clips using our method, executed by physically simulated characters. **Left:** Humanoid character performing a cartwheel. **Right:** Simulated Atlas robot performing a spinkick.

A longstanding goal in character animation is to combine data-driven specification of behavior with a system that can execute a similar behavior in a physical simulation, thus enabling realistic responses to perturbations and environmental variation. We show that well-known reinforcement learning (RL) methods can be adapted to learn robust control policies capable of imitating a broad range of example motion clips, while also learning complex recoveries, adapting to changes in morphology, and accomplishing user-specified goals. Our method handles keyframed motions, highly-dynamic actions such as motion-captured flips and spins, and retargeted motions. By combining a motion-imitation objective with a task objective, we can train characters that react intelligently in interactive settings, e.g., by walking in a desired direction or throwing a ball at a user-specified target. This approach thus combines the convenience and motion quality of using motion clips to define the desired style and appearance, with the flexibility and generality afforded by RL methods and physics-based animation. We further explore a number of methods for integrating multiple clips into the learning process to develop multi-skilled agents capable of performing a rich repertoire of diverse skills. We demonstrate results using multiple characters (human, Atlas robot, bipedal dinosaur, dragon) and a large variety of skills, including locomotion, acrobatics, and martial arts.

CCS Concepts: • **Computing methodologies** → **Animation**; *Physical simulation*; *Control methods*; *Reinforcement learning*;

Additional Key Words and Phrases: physics-based character animation, motion control, reinforcement learning

ACM Reference Format:
Xue Bin Peng, Pieter Abbeel, Sergey Levine, and Michiel van de Panne. 2018. DeepMimic: Example-Guided Deep Reinforcement Learning of Physics-Based Character Skills. *ACM Trans. Graph.* 37, 4, Article 143 (August 2018), 18 pages. https://doi.org/10.1145/3197517.3201311

Authors' addresses: Xue Bin Peng, University of California, Berkeley; Pieter Abbeel, University of California, Berkeley; Sergey Levine, University of California, Berkeley; Michiel van de Panne, University of British Columbia.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
© 2018 Copyright held by the owner/author(s). Publication rights licensed to ACM.
0730-0301/2018/8-ART143 $15.00
https://doi.org/10.1145/3197517.3201311

1 INTRODUCTION

Physics-based simulation of passive phenomena, such as cloth and fluids, has become nearly ubiquitous in industry. However, the adoption of physically simulated characters has been more modest. Modeling the motion of humans and animals remains a challenging problem, and currently, few methods exist that can simulate the diversity of behaviors exhibited in the real world. Among the enduring challenges in this domain are generalization and directability. Methods that rely on manually designed controllers have produced compelling results, but their ability to generalize to new skills and new situations is limited by the availability of human insight. Though humans are adept at performing a wide range of skills themselves, it can be difficult to articulate the internal strategies that underly such proficiency, and more challenging still to encode them into a controller. Directability is another obstacle that has impeded the adoption of simulated characters. Authoring motions for simulated characters remains notoriously difficult, and current interfaces still cannot provide users with an effective means of eliciting the desired behaviours from simulated characters.

Reinforcement learning (RL) provides a promising approach for motion synthesis, whereby an agent learns to perform various skills through trial-and-error, thus reducing the need for human insight. While deep reinforcement learning has been demonstrated to produce a range of complex behaviors in prior work [Duan et al. 2016; Heess et al. 2016; Schulman et al. 2015b], the quality of the generated motions has thus far lagged well behind state-of-the-art kinematic methods or manually designed controllers. In particular, controllers trained with deep RL exhibit severe (and sometimes humorous) artifacts, such as extraneous upper body motion, peculiar gaits, and unrealistic posture [Heess et al. 2017].[1] A natural direction to improve the quality of learned controllers is to incorporate motion capture or hand-authored animation data. In prior work, such systems have typically been designed by layering a physics-based tracking controller on top of a kinematic animation system [Da Silva et al. 2008; Lee et al. 2010a]. This type of approach is challenging because the kinematic animation system must produce reference motions that

[1] See, for example, https://youtu.be/hx_bgoTF7bs

DeepMimic：基于物理的角色技能的示例引导深度强化学习

彭雪斌，加州大学伯克利分校；阿贝尔，加州大学伯克利分校；利文，加州大学伯克利分校；范德潘，英属哥伦比亚大学

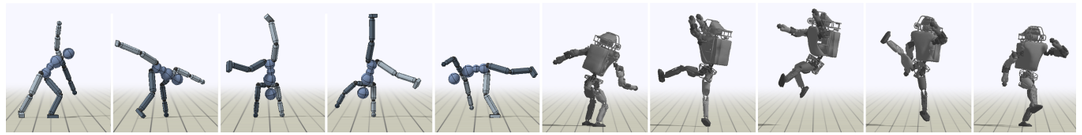


图1。使用我们的方法通过模仿参考运动捕捉片段学习的高动态技能，由物理模拟角色执行。**左：** 人形角色执行前滚翻。**右：** 模拟的Atlas机器人执行旋转踢。

角色动画中的一个长期目标是结合数据驱动的行为指定与能够在一个物理模拟中执行类似行为的系统，从而实现对干扰和环境变化的逼真响应。我们表明，众所周知的强化学习（RL）方法可以适应学习能够模仿广泛示例运动片段的鲁棒控制策略，同时学习复杂的恢复、适应形态变化并实现用户指定的目标。我们的方法处理关键帧运动、高动态动作（如运动捕捉的翻转和旋转）和重定向运动。通过结合运动模仿目标与任务目标，我们可以训练在交互环境中智能反应的角色，例如，通过向期望方向行走或在用户指定的目标处投掷球。这种方法因此结合了使用运动片段定义所需风格和外观的便利性和运动质量，以及强化学习方法基于物理的动画所赋予的灵活性和通用性。我们进一步探索了将多个片段集成到学习过程中以开发能够执行丰富多样的技能的多技能智能体的多种方法。我们使用多个角色（人类、Atlas机器人、两足恐龙、龙）和大量技能（包括运动、杂技和武术）展示了结果。

CCS概念： • **计算方法** → **动画**； 物理模拟； 控制方法； 强化学习；

其他关键词和短语：基于物理的角色动画，运动控制，强化学习

ACM参考文献格式：
彭雪斌、Pieter Abbeel、Sergey Levine 和 Michiel van de Panne。2018。DeepMimic：基于示例指导的物理驱动角色技能深度强化学习。*ACM Trans. Graph.*37, 4, 第143篇文章（2018年8月），18页。
https://doi.org/10.1145/3197517.3201311

作者地址：彭雪斌，加州大学伯克利分校；Pieter Abbeel，加州大学伯克利分校；Sergey Levine，加州大学伯克利分校；Michiel van de Panne，英属哥伦比亚大学。

允许为个人或课堂使用复制本文的全部或部分内容，无需付费，但复制件不得以营利或商业优势为目的制作或分发，并且复制件必须包含本声明并在第一页上注明完整引用。本文中由非作者拥有的组件的版权必须得到尊重。经注明出处后允许摘录。否则复制或再出版、在服务器上发布或重新分发到列表中，需要事先获得特定许可和/或付费。请向permissions@acm.org申请许可。© 2018 版权归所有者/作者所有。出版权许可给ACM。
0730-0301/2018/8-ART143 $15.00
https://doi.org/10.1145/3197517.3201311

1 简介

基于物理的被动现象（如布料和流体）的模拟在工业中已近乎无处不在。然而，基于物理模拟的角色采用则较为保守。模拟人类和动物的运动会仍然是一个具有挑战性的问题，目前，很少有方法能够模拟现实世界中展现出的行为多样性。在这个领域持久的挑战包括泛化和可控性。依赖手动设计控制器的方

强化学习（强化学习）为运动合成提供了一种有前景的方法，智能体通过试错学习执行各种技能，从而减少对人类洞察力的需求。虽然深度强化学习在先前工作中已被证明能够产生一系列复杂行为 [Duan 等人 2016；Heess 等人 2016；Schulman 等人 2015b],，但生成的运动质量迄今为止远远落后于最先进的运动学方法或手动设计的控制器。特别是，使用深度强化学习训练的控制器表现出严重的（有时甚至是滑稽的）伪影，例如额外的上半身运动、奇怪的步态和不真实的姿势 [Heess 等人 2017].[1] 提高学习控制器质量的自然方向是结合动作捕捉或手工制作的动画数据。在先前工作中，此类系统通常是通过在运动学动画系统之上叠加基于物理的跟踪控制器来设计的 [Da Silva 等人 2008； Lee 等人 2010a]。这种方法具有挑战性，因为运动学动画系统必须产生可供跟踪的参考运动，并且生成的基于物理的控制器在修改运动以实现合理的恢复或以与运动学运动显著不同的方式完成任务目标方面的能力有限。此外，此类方法往往实现起来相当复杂。

[1] 例如，请参见 https://youtu.be/hx_bgoTF7bs

are feasible to track, and the resulting physics-based controller is limited in its ability to modify the motion to achieve plausible recoveries or accomplish task goals in ways that deviate substantially from the kinematic motion. Furthermore, such methods tend to be quite complex to implement.

An ideal learning-based animation system should allow an artist or motion capture actor to supply a set of reference motions for style, and then generate goal-directed and physically realistic behavior from those reference motions. In this work, we take a simple approach to this problem by directly rewarding the learned controller for producing motions that resemble reference animation data, while also achieving additional task objectives. We also demonstrate three methods for constructing controllers from multiple clips: training with a multi-clip reward based on a max operator; training a policy to perform multiple diverse skills that can be triggered by the user; and sequencing multiple single-clip policies by using their value functions to estimate the feasibility of transitions.

The central contribution of our paper is a framework for physics-based character animation that combines goal-directed reinforcement learning with data, which may be provided in the form of motion capture clips or keyframed animations. Although our framework consists of individual components that have been known for some time, the particular combination of these components in the context of data-driven and physics-based character animation is novel and, as we demonstrate in our experiments, produces a wide range of skills with motion quality and robustness that substantially exceed prior work. By incorporating motion capture data into a phase-aware policy, our system can produce physics-based behaviors that are nearly indistinguishable in appearance from the reference motion in the absence of perturbations, avoiding many of the artifacts exhibited by previous deep reinforcement learning algorithms, e.g., [Duan et al. 2016]. In the presence of perturbations or modifications, the motions remain natural, and the recovery strategies exhibit a high degree of robustness without the need for human engineering. To the best of our knowledge, we demonstrate some of the most capable physically simulated characters produced by learning-based methods. In our ablation studies, we identify two specific components of our method, reference state initialization and early termination, that are critical for achieving highly dynamic skills. We also demonstrate several methods for integrating multiple clips into a single policy.

2 RELATED WORK

Modeling the skilled movement of articulated figures has a long history in fields ranging from biomechanics to robotics and animation. In recent years, as machine learning algorithms for control have matured, there has also been an increase in interest in these problems from the machine learning community. Here we focus on the most closely related work in animation and RL.

Kinematic Models: Kinematic methods have been an enduring avenue of work in character animation that can be effective when large amounts of data are available. Given a dataset of motion clips, controllers can be built to select the appropriate clip to play in a given situation, e.g., [Agrawal and van de Panne 2016; Lee et al. 2010b; Safonova and Hodgins 2007]. Gaussian processes have been used to learn latent representations which can then synthesize motions at runtime [Levine et al. 2012; Ye and Liu 2010b]. Extending this line of work, deep learning models, such as autoencoders and phase-functioned networks, have also been applied to develop generative models of human motion in a kinematic setting [Holden et al. 2017, 2016]. Given high quality data, data-driven kinematic methods will often produce higher quality motions than most simulation-based approaches. However, their ability to synthesize behaviors for novel situations can be limited. As tasks and environments become complex, collecting enough motion data to provide sufficient coverage of the possible behaviors quickly becomes untenable. Incorporating physics as a source of prior knowledge about how motions should change in the presence of perturbations and environmental variation provides one solution to this problem, as discussed below.

Physics-based Models: Design of controllers for simulated characters remains a challenging problem, and has often relied on human insight to implement task-specific strategies. Locomotion in particular has been the subject of considerable work, with robust controllers being developed for both human and nonhuman characters, e.g., [Coros et al. 2010; Ye and Liu 2010a; Yin et al. 2007]. Many such controllers are the products of an underlying simplified model and an optimization process, where a compact set of parameters are tuned in order to achieve the desired behaviors [Agrawal et al. 2013; Ha and Liu 2014; Wang et al. 2012]. Dynamics-aware optimization methods based on quadratic programming have also been applied to develop locomotion controllers [Da Silva et al. 2008; Lee et al. 2010a, 2014]. While model-based methods have been shown to be effective for a variety of skills, they tend to struggle with more dynamics motions that require long-term planning, as well as contact-rich motions. Trajectory optimization has been explored for synthesizing physically-plausible motions for a variety of tasks and characters [Mordatch et al. 2012; Wampler et al. 2014]. These methods synthesize motions over an extended time horizon using an offline optimization process, where the equations of motion are enforced as constraints. Recent work has extended such techniques into online model-predictive control methods [Hämäläinen et al. 2015; Tassa et al. 2012], although they remain limited in both motion quality and capacity for long-term planning. The principal advantage of our method over the above approaches is that of generality. We demonstrate that a single model-free framework is capable of a wider range of motion skills (from walks to highly dynamic kicks and flips) and an ability to sequence these; the ability to combine motion imitation and task-related demands; compact and fast-to-compute control policies; and the ability to leverage rich high-dimensional state and environment descriptions.

Reinforcement Learning: Many of the optimization techniques used to develop controllers for simulated characters are based on reinforcement learning. Value iteration methods have been used to develop kinematic controllers to sequence motion clips in the context of a given task [Lee et al. 2010b; Levine et al. 2012]. Similar approaches have been explored for simulated characters [Coros et al. 2009; Peng et al. 2015]. More recently, the introduction of deep neural network models for RL has given rise to simulated agents that can perform a diverse array of challenging tasks [Brockman et al. 2016a; Duan et al. 2016; Liu and Hodgins 2017; Peng et al. 2016; Rajeswaran

是可行的，并且基于物理的控制器在修改运动以实现合理的恢复或以与运动学运动显著偏离的方式完成任务目标方面能力有限。此外，此类方法往往实现起来相当复杂。

一个理想的基于学习的动画系统应该允许艺术家或动作捕捉演员提供一组用于风格的参考运动，然后从这些参考运动中生成目标导向且物理上逼真的行为。在这项工作中，我们通过直接奖励学习到的控制器生成与参考动画数据相似的运动的解决方案，同时实现附加的任务目标，采用了一种简单的方法来解决这个问题。我们还展示了三种从多个片段构建控制器的方法：基于最大算子的多片段奖励训练；训练一个策略以执行用户可以触发的多个不同技能；以及通过使用它们的价值函数来估计转换的可行性，依次排列多个单片段策略。

我们论文的核心贡献是一个基于物理的角色动画框架，该框架结合了目标导向的强化学习和数据，数据可以以运动捕捉片段或关键帧动画的形式提供。尽管我们的框架由一些已知组件组成，但这些组件在数据驱动和基于物理的角色动画环境下的特定组合是新颖的，正如我们在实验中所展示的，它产生的运动质量和鲁棒性显著优于先前的工作。通过将运动捕捉数据纳入感知策略，我们的系统可以在没有扰动的情况下生成几乎无法与参考运动区分的基于物理的行为，避免了先前深度强化学习算法表现出许多伪影，例如 [Duan 等人 2016]。在有扰动或修改的情况下，运动仍然自然，恢复策略表现出高度的鲁棒性，无需人工工程。据我们所知，我们展示了基于学习方法产生的最强大的物理模拟角色。在我们的消融研究中，我们确定了我们方法中两个特定的组件，参考状态初始化和提前终止，对于实现高度动态的技能至关重要。我们还展示了几种将多个片段集成到单个策略中的方法。

2 相关工作

模拟关节化角色的熟练运动在从生物力学到机器人学和动画等领域中有着悠久的历史。近年来，随着控制机器学习算法的成熟，机器学习社区对这些问题的兴趣也有所增加。在这里，我们关注与动画和强化学习最密切相关的作品。

运动学模型：运动学方法一直是角色动画领域一个持久的研究方向，当有大量数据可用时，这些方法可以非常有效。给定一个运动片段数据集，可以构建控制器来选择在特定情况下播放的合适片段，例如 [Agrawal 和 van de Panne 2016；Lee 等人 2010b；Safonova 和 Hodgins 2007]。高斯过程已被用于学习可以运行时合成运动的潜在表示 [Levine 等人 2012 年；Ye 和 Liu 2010b]。沿着这条线的工作，深度学习模型，如自动编码器和相位函数网络，也已应用于在运动学环境中开发人体运动的生成模型 [2017, 2016] Holden 等人。。给定高质量数据，数据驱动的运动学方法通常会产生比大多数基于模拟的方法更高质量的运动的。然而，它们为新颖情况合成行为的能力可能有限。随着任务和环境变得复杂，收集足够的运动数据以提供对可能行为的充分覆盖很快变得不可行。将物理作为关于运动如何在扰动和环境变化存在时应该如何变化的先验知识的来源为这个问题提供了一种解决方案，如下所述。

物理模型：为模拟角色设计控制器仍然是一个具有挑战性的问题，并且通常依赖于人类洞察力来实现特定任务的策略。特别是运动方面，一直是大量工作的主题，为人类和非人类角色开发了稳健的控制器，例如 [Coros 等人 2010 年；Ye 和 Liu 2010a 年；Yin 等人 2007]。许多此类控制器是底层简化模型和优化过程的产品，其中一组紧凑的参数被调整以实现所需的行为 [Agrawal 等人 2013 年；Ha 和 Liu 2014 年；Wang 等人 2012]。基于二次规划的动力学感知优化方法也已应用于开发运动控制器 [Da Silva 等人 2008 年；Lee 等人 2010a 年，2014]。虽然基于模型的方法已被证明对各种技能是有效的，但它们往往难以处理需要长期规划的更复杂的动力学运动，以及接触丰富的运动。轨迹优化已被探索用于为各种任务和角色合成物理上合理的运动 [Mordatch 等人 2012 年；Wampler 等人 2014]。这些方法使用离线优化过程在扩展的时间视野内合成运动，其中运动方程被作为约束。最近的工作将此类技术扩展到在线模型预测控制方法 [Hämäläinen 等人 2015 年；Tassa 等人 2012]，，尽管它们在运动质量和长期规划能力方面仍然有限。我们方法相对于上述方法的主要优势在于其通用性。我们证明了一个无模型的框架能够执行更广泛的运动技能（从行走到高度动态的踢和翻转），并能够对它们进行序列化；结合运动模仿和与任务相关的需求的能力；紧凑且快速计算的控制器策略；以及利用丰富的高维状态和环境描述的能力。

强化学习：许多用于开发模拟角色控制器的优化技术都基于强化学习。值迭代方法已被用于开发运动片段序列的动力学控制器，以实现给定任务中的运动 [Lee 等人 2010b；Levine 等人 2012]。类似的方法已被探索用于模拟角色 [Coros 等人 2009；Peng 等人 2015]。最近，深度神经网络模型在强化学习中的应用，催生了能够执行各种复杂任务的模拟智能体 [Brockman 等人 2016a；Duan 等人 2016；Liu 和 Hodgins 2017；Peng 等人 2016；Rajeswaran

et al. 2017; Teh et al. 2017]. Policy gradient methods have emerged as the algorithms of choice for many continuous control problems [Schulman et al. 2015a, 2017; Sutton and Barto 1998]. Although RL algorithms have been capable of synthesizing controllers using minimal task-specific control structures, the resulting behaviors generally appear less natural than their more manually engineered counterparts [Merel et al. 2017; Schulman et al. 2015b]. Part of the challenge stems from the difficulty in specifying reward functions for natural movement, particularly in the absence of biomechanical models and objectives that can be used to achieve natural simulated locomotion [Lee et al. 2014; Wang et al. 2012]. Naïve objectives for torque-actuated locomotion, such as forward progress or maintaining a desired velocity, often produce gaits that exhibit extraneous motion of the limbs, asymmetric gaits, and other objectionable artifacts. To mitigate these artifacts, additional objectives such as effort or impact penalties have been used to discourage these undesirable behaviors. Crafting such objective functions requires a substantial degree of human insight, and often yields only modest improvements. Alternatively, recent RL methods based on the imitation of motion capture, such as GAIL [Ho and Ermon 2016], address the challenge of designing a reward function by using data to induce an objective. While this has been shown to improve the quality of the generated motions, current results still do not compare favorably to standard methods in computer animation [Merel et al. 2017]. The DeepLoco system [Peng et al. 2017a] takes an approach similar to the one we use here, namely adding an imitation term to the reward function, although with significant limitations. It uses fixed initial states and is thus not capable of highly dynamic motions; it is demonstrated only on locomotion tasks defined by foot-placement goals computed by a high-level controller; and it is applied to a single armless biped model. Lastly, the multi-clip demonstration involves a hand-crafted procedure for selecting suitable target clips for turning motions.

Motion Imitation: Imitation of reference motions has a long history in computer animation. An early instantiation of this idea was in bipedal locomotion with planar characters [Sharon and van de Panne 2005; Sok et al. 2007], using controllers tuned through policy search. Model-based methods for tracking reference motions have also been demonstrated for locomotion with 3D humanoid characters [Lee et al. 2010a; Muico et al. 2009; Yin et al. 2007]. Reference motions have also been used to shape the reward function for deep RL to produce more natural locomotion gaits [Peng et al. 2017a,b] and for flapping flight [Won et al. 2017]. In our work, we demonstrate the capability to perform a significantly broader range of difficult motions: highly dynamic spins, kicks, and flips with intermittent ground contact, and we show that reference-state initialization and early termination are critical to their success. We also explore several options for multi-clip integration and skill sequencing.

The work most reminiscent of ours in terms of capabilities is the Sampling-Based Controller (SAMCON) [Liu et al. 2016, 2010]. An impressive array of skills has been reproduced by SAMCON, and to the best of our knowledge, SAMCON has been the only system to demonstrate such a diverse corpus of highly dynamic and acrobatic motions with simulated characters. However, the system is complex, having many components and iterative steps, and requires defining a low dimensional state representation for the synthesized linear feedback structures. The resulting controllers excel at mimicking the original reference motions, but it is not clear how to extend the method for task objectives, particularly if they involve significant sensory input. A more recent variation introduces deep Q-learning to train a high-level policy that selects from a precomputed collection of SAMCON control fragments [Liu and Hodgins 2017]. This provides flexibility in the order of execution of the control fragments, and is demonstrated to be capable of challenging non-terminating tasks, such as balancing on a bongo-board and walking on a ball. In this work, we propose an alternative framework using deep RL, that is conceptually much simpler than SAMCON, but is nonetheless able to learn highly dynamic and acrobatic skills, including those having task objectives and multiple clips.

3 OVERVIEW

Our system receives as input a character model, a corresponding set of kinematic reference motions, and a task defined by a reward function. It then synthesizes a controller that enables the character to imitate the reference motions, while also satisfying task objectives, such as striking a target or running in a desired direction over irregular terrain. Each reference motion is represented as a sequence of target poses $\{\hat{q}_t\}$. A control policy $\pi(a_t|s_t, g_t)$ maps the state of the character s_t, a task-specific goal g_t to an action a_t, which is then used to compute torques to be applied to each of the character's joints. Each action specifies target angles for proportional-derivative (PD) controllers that then produce the final torques applied at the joints. The reference motions are used to define an imitation reward $r^I(s_t, a_t)$, and the goal defines a task-specific reward $r^G(s_t, a_t, g_t)$. The final result of our system is a policy that enables a simulated character to imitate the behaviours from the reference motions while also fulfilling the specified task objectives. The policies are modeled using neural networks and trained using the proximal policy optimization algorithm [Schulman et al. 2017].

4 BACKGROUND

Our tasks will be structured as standard reinforcement learning problems, where an agent interacts with an environment according to a policy in order to maximize a reward. In the interest of brevity, we will exclude the goal g, from the notation, but the following discussion readily generalizes to include this. A policy $\pi(a|s)$ models the conditional distribution over action $a \in A$ given a state $s \in S$. At each control timestep, the agent observes the current state s_t and samples an action a_t from π. The environment then responds with a new state $s' = s_{t+1}$, sampled from the dynamics $p(s'|s, a)$, and a scalar reward r_t that reflects the desirability of the transition. For a parametric policy $\pi_\theta(a|s)$, the goal of the agent is to learn the optimal parameters θ^* that maximizes its expected return

$$J(\theta) = \mathbb{E}_{\tau \sim p_\theta(\tau)} \left[\sum_{t=0}^{T} \gamma^t r_t \right],$$

where $p_\theta(\tau) = p(s_0) \prod_{t=0}^{T-1} p(s_{t+1}|s_t, a_t)\pi_\theta(a_t|s_t)$ is the distribution over all possible trajectories $\tau = (s_0, a_0, s_1, ..., a_{T-1}, s_T)$ induced by the policy π_θ, with $p(s_0)$ being the initial state distribution. $\sum_{t=0}^{T} \gamma^t r_t$ represents the total return of a trajectory, with a horizon

et al. 2017；Teh 等人 2017]。策略梯度方法已成为许多连续控制问题的首选算法[Schulman 等人 2015a，2017；Sutton 和 Barto 1998]。尽管强化学习算法能够使用最少的特定任务控制结构来合成控制器，但生成的行为通常不如更手动设计的对应物自然 [Merel 等人 2017；Schulman 等人 2015b]。部分挑战源于指定自然运动奖励函数的困难，特别是在缺乏可用于实现自然模拟运动的生物力学模型和目标的情况下 [Lee 等人 2014；Wang 等人 2012]。对于由扭矩驱动的运动，如向前推进或保持所需速度等简单目标，往往会产生肢体多余运动、非对称步态和其他令人不快的伪影。为了减轻这些伪影，已使用额外的目标（如努力或冲击惩罚）来阻止这些不希望的行为。设计此类目标函数需要相当程度的人类洞察力，并且通常只能带来适度的改进。或者，基于动作捕捉模仿的最近强化学习方法，如 GAIL [Ho 和 Ermon 2016]，通过使用数据来诱导目标，从而解决了设计奖励函数的挑战。虽然这已被证明可以提高生成运动的质量，但当前结果与计算机动画中的标准方法相比仍不占优势 [Merel 等人 2017]。DeepLoco 系统 [Peng 等人 2017a] 采用了与这里使用的方法相似的策略，即在奖励函数中添加模仿项，尽管存在显著限制。它使用固定初始状态，因此无法执行高度动态的运动；它仅在由高级控制器计算的足部放置目标定义的运动任务中演示；并且它应用于单个无臂双足模型。最后，多片段演示涉及为转向运动选择合适目标片段的手工过程。

运动模仿：模仿参考运动在计算机动画中有着悠久的历史。这个想法的早期实例出现在使用平面角色的双足运动中 [Sharon 和 van de Panne 2005；Sok 等人 2007], 使用通过策略搜索调优的控制器。基于模型的跟踪参考运动的方法也已被展示用于具有 3D 人形机器人的运动 [Lee 等人 2010a；Muico 等人 2009；Yin 等人 2007]。参考运动还用于为深度强化学习塑造奖励函数，以产生更自然的运动步态 [Peng 等人 2017a,b] 以及用于扑翼飞行 [Won 等人 2017]。在我们的工作中，我们展示了执行更广泛范围困难运动的能力：高度动态的旋转、踢和间歇性接触地面的翻转，并表明参考状态初始化和提前终止对其成功至关重要。我们还探索了多片段集成和技能排序的几种选项。

在能力方面与我们最相似的工作是采样控制器（SAMCON）[刘等人 2016, 2010]。SAMCON已经复现了一系列令人印象深刻的技能，据我们所知，SAMCON是目前唯一一个能够用模拟角色展示如此多样化、高度动态和杂技性运动的系统。然而，该系统非常复杂，包含许多组件和迭代步骤，并且需要定义合成线性反馈结构的低维状态表示。生成的控制器擅长模仿原始参考运动，但尚不清楚如何将该方法扩展到任务目标，特别是当它们涉及大量感官输入时。一个更新的变体引入了深度Q学习来训练一个高级策略，该策略从预先计算的SAMCON控制片段集合 [刘和霍金 2017]中选择。这提供了控制片段执行顺序的灵活性，并且已被证明能够应对具有挑战性的非终止任务，例如在邦戈板上平衡和在球上行走。在这项工作中，我们提出了一种使用深度RL的替代框架，它在概念上比SAMCON简单得多，但仍然能够学习高度动态和杂技性的技能，包括具有任务目标和多个片段的技能。

3 概述

我们的系统接收一个角色模型、一组对应的运动学参考运动以及由奖励函数定义的任务作为输入。然后，它合成一个控制器，使角色能够模仿参考运动，同时满足任务目标，例如击中目标或在非规则地形上朝期望方向跑步。每个参考运动表示为一系列目标姿态 $\{\hat{q}_t\}$。控制策略 $\pi(a_t|s_t, g_t)$ 将角色的状态 s_t 和特定任务目标 g_t映射到一个动作 a_t，然后用于计算应用于角色每个关节的扭矩。每个动作指定用于比例-微分 (PD) 控制器的目标角度，这些控制器随后产生应用于关节的最终扭矩。参考运动用于定义模仿奖励 $r_I(s_t, a_t)$，而目标定义特定任务奖励 $r_G(s_t, a_t, g_t)$。我们系统的最终结果是一个策略，使模拟角色能够模仿参考运动的行为，同时满足指定的任务目标。策略使用神经网络建模，并使用近端策略优化算法 [Schulman et al. 2017]进行训练。

4 背景

我们的任务将被结构化为标准的强化学习问题，其中智能体根据策略与环境交互以最大化奖励。为了简洁起见，我们将从符号中排除目标 g，但以下讨论可以很容易地推广以包含它。策略 $\pi(a|s)$ 对给定状态 $s \in S$ 的动作 $a \in A$ 的条件分布进行建模。在每个控制时间步，智能体观察当前状态 s_t 并从 π 中采样一个动作 a_t。环境随后根据动力学 $p(s_t|s,a)$ 采样一个新的状态 $s_t = s_{t+1}$ 并返回一个标量奖励 r_t，该奖励反映了转换的期望性。对于参数化策略 $\pi_\theta(a|s)$，智能体的目标是学习使预期回报最大化的最优参数 θ^*。

$$J(\theta) = \mathbb{E}_{\tau \sim p_\theta(\tau)} \left[\sum_{t=0}^{T} \gamma^t r_t \right],$$

其中 $p_\theta(\tau) = p(s0) \prod_{t=0}^{T-1} p(s_{t+1}|s_t, a_t)\pi_\theta(a_t|s_t)$ 是策略 π_θ 所产生的所有可能轨迹 $\tau = (s0, a0, s1, ..., aT-1, sT)$ 上的分布，$p(s0)$ 为初始状态分布。$\sum_{t=0}^{T} \gamma^t r_t$ 表示轨迹的总回报，其中 T 为视野

of T steps. T may or may not be infinite, and $\gamma \in [0, 1]$ is a discount factor that can be used to ensure the return is finite. A popular class of algorithms for optimizing a parametric policy is policy gradient methods [Sutton et al. 2001], where the gradient of the expected return $\nabla_\theta J(\theta)$ is estimated with trajectories sampled by following the policy. The policy gradient can be estimated according to

$$\nabla_\theta J(\theta) = \mathbb{E}_{s_t \sim d_\theta(s_t), a_t \sim \pi_\theta(a_t|s_t)} \left[\nabla_\theta \log(\pi_\theta(a_t|s_t)) \mathcal{A}_t \right],$$

where $d_\theta(s_t)$ is the state distribution under the policy π_θ. $\mathcal{A}_t$ represents the advantage of taking an action a_t at a given state s_t

$$\mathcal{A}_t = R_t - V(s_t).$$

$R_t = \sum_{l=0}^{T-t} \gamma^l r_{t+l}$ denotes the return received by a particular trajectory starting from state s_t at time t. $V(s_t)$ is a value function that estimates the average return of starting in s_t and following the policy for all subsequent steps

$$V(s_t) = \mathbb{E}\left[R_t | \pi_\theta, s_t\right].$$

The policy gradient can therefore be interpreted as increasing the likelihood of actions that lead to higher than expected returns, while decreasing the likelihood of actions that lead to lower than expected returns. A classic policy gradient algorithm for learning a policy using this empirical gradient estimator to perform gradient ascent on $J(\theta)$ is REINFORCE [Williams 1992].

Our policies will be trained using the proximal policy optimization algorithm [Schulman et al. 2017], which has demonstrated state-of-the-art results on a number of challenging control problems. The value function will be trained using multi-step returns with TD(λ). The advantages for the policy gradient will be computed using the generalized advantage estimator GAE(λ) [Schulman et al. 2015b]. A more in-depth review of these methods can be found in the supplementary material.

5 POLICY REPRESENTATION

Given a reference motion clip, represented by a sequence of target poses $\{\hat{q}_t\}$, the goal of the policy is to reproduce the desired motion in a physically simulated environment, while also satisfying additional task objectives. Since a reference motion only provides kinematic information in the form of target poses, the policy is responsible for determining which actions should be applied at each timestep in order to realize the desired trajectory.

5.1 States and Actions

The state s describes the configuration of the character's body, with features consisting of the relative positions of each link with respect to the root (designated to be the pelvis), their rotations expressed in quaternions, and their linear and angular velocities. All features are computed in the character's local coordinate frame, with the root at the origin and the x-axis along the root link's facing direction. Since the target poses from the reference motions vary with time, a phase variable $\phi \in [0, 1]$ is also included among the state features. $\phi = 0$ denotes the start of a motion, and $\phi = 1$ denotes the end. For cyclic motions, ϕ is reset to 0 after the end of each cycle. Policies trained to achieve additional task objectives, such as walking in a particular direction or hitting a target, are also provided with a goal g, which can be treated in a similarly fashion as the state. Specific goals used in the experiments are discussed in section 9. The action a from the policy specifies target orientations for PD controllers at each joint. The policy is queried at 30Hz, and target orientations for spherical joints are represented in axis-angle form, while targets for revolute joints are represented by scalar rotation angles. Unlike the standard benchmarks, which often operate directly on torques, our use of PD controllers abstracts away low-level control details such as local damping and local feedback. Compared to torques, PD controllers have been shown to improve performance and learning speed for certain motion control tasks [Peng et al. 2017b].

5.2 Network

Each policy π is represented by a neural network that maps a given state s and goal g to a distribution over action $\pi(a|s, g)$. The action distribution is modeled as a Gaussian, with a state dependent mean $\mu(s)$ specified by the network, and a fixed diagonal covariance matrix Σ that is treated as a hyperparameter of the algorithm:

$$\pi(a|s) = \mathcal{N}(\mu(s), \Sigma).$$

The inputs are processed by two fully-connected layers with 1024, and 512 units each, followed by a linear output layer. ReLU activations are used for all hidden units. The value function is modeled by a similar network, with exception of the output layer, which consists of a single linear unit.

For vision-based tasks, discussed in section 9, the inputs are augmented with a heightmap H of the surrounding terrain, sampled on a uniform grid around the character. The policy and value networks are augmented accordingly with convolutional layers to process the heightmap. A schematic illustration of this visuomotor policy network is shown in Figure 2. The heightmap is first processed by a series of convolutional layers, followed by a fully-connected layer. The resulting features are then concatenated with the input state s and goal g, and processed by a similar fully-connected network as the one used for tasks that do not require vision.

5.3 Reward

The reward r_t at each step t consists of two terms that encourage the character to match the reference motion while also satisfying

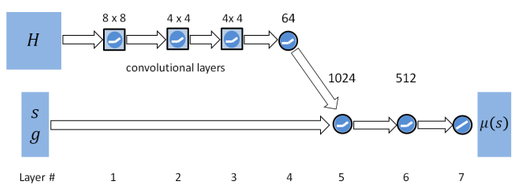


Fig. 2. Schematic illustration of the visuomotor policy network. The heightmap H is processed by 3 convolutional layers with 16 8x8 filters, 32 4x4 filters, and 32 4x4 filters. The feature maps are then processed by 64 fully-connected units. The resulting features are concatenated with the input state s and goal g and processed by by two fully-connected layer with 1024 and 512 units. The output $\mu(s)$ is produced by a layer of linear units. ReLU activations are used for all hidden layers. For tasks that do not require a heightmap, the networks consist only of layers 5-7.

包含 T步。 T可能为有限也可能为无限， $\gamma \in [0, 1]$ 是一个折扣因子，可用于确保回报为有限值。用于优化参数化策略的流行算法类别是策略梯度方法 [Sutton et al. 2001]，其中通过遵循策略采样的轨迹来估计预期回报 $\nabla_\theta J(\theta)$ 的梯度。策略梯度可以根据以下方式估计

$$\nabla_\theta J(\theta) = \mathbb{E}_{s_t \sim d_\theta(s_t), a_t \sim \pi_\theta(a_t|s_t)} \left[\nabla_\theta \log(\pi_\theta(a_t|s_t)) \mathcal{A}_t \right],$$

其中 $d_\theta(s_t)$ 是策略 π_θ 下的状态分布。A$_t$表示在给定状态s_t下采取动作 a_t的优势

$$\mathcal{A}_t = R_t - V(s_t).$$

$R_t = \sum_{T-t l=0} \gamma_{l r t+l}$表示从时间 t时的状态 s_t开始的特定轨迹所获得的回报。$V(s_t)$ 是一个值函数，用于估计从状态 s_t 开始并遵循策略进行所有后续步骤的平均回报

$$V(s_t) = \mathbb{E}\left[R_t | \pi_\theta, s_t\right].$$

因此，策略梯度可以解释为增加那些导致高于预期回报的动作的likelihood，同时减少那些导致低于预期回报的动作的likelihood。一种经典的策略梯度算法，使用这种经验梯度估计器在 $J(\theta)$ 上进行梯度上升，是 REINFORCE [Williams 1992]

我们的策略将使用近端策略优化算法 [Schulman 等人 2017]，进行训练，该算法在多个具有挑战性的控制问题上展示了最先进的结果。值函数将使用多步回报和 TD(λ) 进行训练。策略梯度的优势将使用广义优势估计器 GAE(λ) [Schulman 等人 2015b] 计算。这些方法的更深入综述可以在补充材料中找到。

5 策略表示

给定一个参考运动片段，由一系列目标姿态 { ^<样式 id='1'>q</样式><样式 id='3'>t </样式>} 表示，策略的目标是在物理模拟环境中重现期望的运动，同时满足附加的任务目标。由于参考运动仅以目标姿态的形式提供运动学信息，因此策略负责确定在每个时间步长应用哪些动作，以实现期望的轨迹。

5.1 状态和动作

状态s 描述了角色身体的配置，其特征由每个连杆相对于根（指定为骨盆）的相对位置、用四元数表示的旋转以及它们的线性和角速度组成。所有特征都在角色的局部坐标系中计算，根位于原点，x轴沿根连杆的朝向方向。由于参考运动的靶标姿态随时间变化，一个相位变量 $\phi \in [0, 1]$ 也被包含在状态特征中。$\phi = 0$ 表示运动的开始， $\phi = 1$ 表示结束。对于循环运动，ϕ 在每个循环结束后重置为0。为了实现额外的任务目标（例如朝特定方向行走或击中目标），训练的策略也会提供一个目标 g，它可以与状态类似地处理。具体

实验中使用的目标在第九节中讨论。策略中的动作a 指定了每个关节的比例-微分控制器（PD controllers）的目标方向。策略以30Hz的频率被查询，球关节的目标方向以轴-角形式表示，而旋转关节的目标则由标量旋转角表示。与通常直接操作扭矩的标准基准不同，我们使用比例-微分控制器（PD controllers）抽象了低级控制细节，例如局部阻尼和局部反馈。与扭矩相比，比例-微分控制器（PD controllers）已被证明可以提高某些运动控制任务的性能和学习速度 [Peng et al. 2017b]。

5.2 网络

每个策略 π 由一个神经网络表示，该网络将给定状态 <样式 id='2'>s </样式>和目标 <样式 id='4'>д </样式>映射到动作 π （<样式 id='7'>a</样式>|<样式 id='10'>s</样式>, <样式 id='12'>д</样式>）的分布。动作分布被建模为高斯分布，其状态相关的均值 <样式 id='14'>μ</样式>(<样式 id='16'>s</样式>) 由网络指定，并且具有一个固定的对角协方差矩阵Σ，该矩阵被视为算法的超参数：

$$\pi(a|s) = \mathcal{N}(\mu(s), \Sigma).$$

输入由两个全连接层处理，每个层分别具有 1024 和 512 个单元，然后是一个线性输出层。所有隐藏单元使用 ReLU 激活函数。值函数由一个类似的网络建模，除外输出层，该层由一个线性单元组成。

对于基于视觉的任务（如第9节所述），输入会通过周围地形的高度图H进行增强，该高度图以角色为中心在均匀网格上进行采样。策略网络和价值网络会相应地通过卷积层来处理高度图。这种视觉运动策略网络的示意图如图2所示。高度图首先通过一系列卷积层进行处理，然后接一个全连接层。生成的特征随后与输入状态s和目标g连接，并由一个与用于不需要视觉的任务的相同类型的全连接网络进行处理。

5.3 奖励

每一步r_t 的奖励由两项组成，这两项鼓励角色匹配参考运动，同时满足

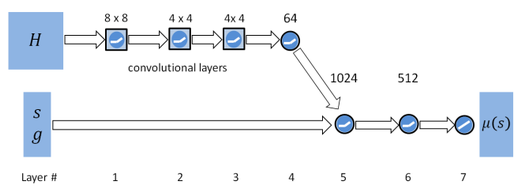


图2. 视觉运动策略网络的示意图。高度图 H通过3个卷积层进行处理，分别为16个8x8滤波器、32个4x4滤波器和32个4x4滤波器。特征图随后通过64个全连接单元进行处理。生成的特征与输入状态 s和目标g连接，并由两个全连接层进行处理，分别为1024和512个单元。输出$\mu(s)$由一个线性单元层产生。所有隐藏层都使用ReLU激活函数。对于不需要高度图的任务，网络仅由第5-7层组成。

additional task objectives:

$$r_t = \omega^I r_t^I + \omega^G r_t^G.$$

Here, r_t^I and r_t^G represent the imitation and task objectives, with ω^I and ω^G being their respective weights. The task objective r_t^G incentivizes the character to fulfill task-specific objectives, the details of which will be discussed in the following section. The imitation objective r_t^I encourages the character to follow a given reference motion $\{\hat{q}_t\}$. It is further decomposed into terms that reward the character for matching certain characteristics of the reference motion, such as joint orientations and velocities, as follows:

$$r_t^I = w^p r_t^p + w^v r_t^v + w^e r_t^e + w^c r_t^c$$
$$w^p = 0.65, w^v = 0.1, w^e = 0.15, w^c = 0.1$$

The pose reward r_t^p encourages the character to match the joint orientations of the reference motion at each step, and is computed as the difference between the joint orientation quaternions of the simulated character and those of the reference motion. In the equation below, q_t^j and $\hat{q}_t^j$ represent the orientations of the jth joint from the simulated character and reference motion respectively, $q_1 \ominus q_2$ denotes the quaternion difference, and $||q||$ computes the scalar rotation of a quaternion about its axis in radians:

$$r_t^p = \exp\left[-2\left(\sum_j ||\hat{q}_t^j \ominus q_t^j||^2\right)\right].$$

The velocity reward r_t^v is computed from the difference of local joint velocities, with $\dot{q}_t^j$ being the angular velocity of the jth joint. The target velocity $\hat{\dot{q}}_t^j$ is computed from the data via finite difference.

$$r_t^v = \exp\left[-0.1\left(\sum_j ||\hat{\dot{q}}_t^j - \dot{q}_t^j||^2\right)\right].$$

The end-effector reward r_t^e encourages the character's hands and feet to match the positions from the reference motion. Here, p_t^e denotes the 3D world position in meters of end-effector $e \in$ [left foot, right foot, left hand, right hand]:

$$r_t^e = \exp\left[-40\left(\sum_e ||\hat{p}_t^e - p_t^e||^2\right)\right].$$

Finally, r_t^c penalizes deviations in the character's center-of-mass p_t^c from that of the reference motion $\hat{p}_t^c$:

$$r_t^c = \exp\left[-10\left(||\hat{p}_t^c - p_t^c||^2\right)\right].$$

6 TRAINING

Our policies are trained with PPO using the clipped surrogate objective [Schulman et al. 2017]. We maintain two networks, one for the policy $\pi_\theta(a|s,g)$ and another for the value function $V_\psi(s,g)$, with parameters θ and ψ respectively. Training proceeds episodically, where at the start of each episode, an initial state s_0 is sampled uniformly from the reference motion (section 6.1), and rollouts are generated by sampling actions from the policy at every step. Each episode is simulated to a fixed time horizon or until a termination condition has been triggered (section 6.2). Once a batch of data has been collected, minibatches are sampled from the dataset and used to update the policy and value function. The value function is updated using target values computed with TD(λ) [Sutton and Barto 1998]. The policy is updated using gradients computed from the surrogate objective, with advantages $\mathcal{A}_t$ computed using GAE(λ) [Schulman et al. 2015b]. Please refer to the supplementary material for a more detailed summary of the learning algorithm.

One of the persistent challenges in RL is the problem of exploration. Since most formulations assume an unknown MDP, the agent is required to use its interactions with the environment to infer the structure of the MDP and discover high value states that it should endeavor to reach. A number of algorithmic improvements have been proposed to improve exploration, such as using metrics for novelty or information gain [Bellemare et al. 2016; Fu et al. 2017; Houthooft et al. 2016]. However, less attention has been placed on the structure of the episodes during training and their potential as a mechanism to guide exploration. In the following sections, we consider two design decisions, the initial state distribution and the termination condition, which have often been treated as fixed properties of a given RL problem. We will show that appropriate choices are crucial for allowing our method to learn challenging skills such as highly-dynamic kicks, spins, and flips. With common default choices, such as a fixed initial state and fixed-length episodes, we find that imitation of these difficult motions is often unsuccessful.

6.1 Initial State Distribution

The initial state distribution $p(s_0)$ determines the states in which an agent begins each episode. A common choice for $p(s_0)$ is to always place the agent in a fixed state. However, consider the task of imitating a desired motion. A simple strategy is to initialize the character to the starting state of the motion, and allow it to proceed towards the end of the motion over the course of an episode. With this design, the policy must learn the motion in a sequential manner, by first learning the early phases of the motion, and then incrementally progressing towards the later phases. Before mastering the earlier phases, little progress can be made on the later phases. This can be problematic for motions such as backflips, where learning the landing is a prerequisite for the character to receive a high return from the jump itself. If the policy cannot land successfully, jumping will actually result in *worse* returns. Another disadvantage of a fixed initial state is the resulting exploration challenge. The policy only receives reward retrospectively, once it has visited a state. Therefore, until a high-reward state has been visited, the policy has no way of learning that this state is favorable. Both disadvantages can be mitigated by modifying the initial state distribution.

For many RL tasks, a fixed initial state can be more convenient, since it can be challenging to initialize the agent in other states (e.g., physical robots) or obtain a richer initial state distribution. For motion imitation tasks, however, the reference motion provides a rich and informative state distribution, that can be leveraged to guide the agent during training. At the start of each episode, a state can be sampled from the reference motion, and used to initialize the state of the agent. We will refer to this strategy as *reference state initialization* (RSI). Similar strategies have been previously used for planar bipedal walking [Sharon and van de Panne 2005] and manipulation [Nair et al. 2017; Rajeswaran et al. 2017]. By sampling initial

附加的任务目标：

$$r_t = \omega^I r_t^I + \omega^G r_t^G.$$

这里，r它和 r_{Gt} 分别表示模仿目标和任务目标，ω_I 和 ω_G 是它们各自的权重。任务目标 r_{Gt} 鼓励角色完成特定的任务目标，其细节将在下一节中讨论。模仿目标 r_{It} 鼓励角色遵循给定的参考运动 $\{\hat{}q_t\}$。它进一步分解为奖励角色匹配参考运动某些特征的项，例如关节方向和速度，如下所示：

$$r_t^I = w^p r_t^p + w^v r_t^v + w^e r_t^e + w^c r_t^c$$
$$w^p = 0.65, w^v = 0.1, w^e = 0.15, w^c = 0.1$$

姿态奖励$r_{p\ t}$ 鼓励角色在每个步骤中匹配参考运动的关节方向，并计算为模拟角色的关节方向四元数与参考运动关节方向四元数之间的差值。在下面的公式中，$q_{j\ t}$ 和 $\hat{}q_{j\ t}$ 分别表示模拟角色和参考运动的第 j 个关节的方向，$q_1 \ominus q_2$ 表示四元数差值，$||q||$ 计算四元数绕其轴旋转的标量旋转，单位为弧度：

$$r_t^p = \exp\left[-2\left(\sum_j ||\hat{q}_t^j \ominus q_t^j||^2\right)\right].$$

速度奖励r_{vt}是根据局部关节速度的差值计算的，其中 $\dot{}q_{j\ t}$ 是第 j 个关节的角速度。目标速度 $\hat{}\dot{}q_{j\ t}$ 是通过有限差分从数据中计算的。

$$r_t^v = \exp\left[-0.1\left(\sum_j ||\hat{\dot{q}}_t^j - \dot{q}_t^j||^2\right)\right].$$

末端执行器奖励r_{et} 鼓励角色的手和脚匹配参考运动的位姿。在这里，p_{et}表示末端执行器$e \in$ [左脚、右脚、左手、右手]在米制单位下的3D世界位置：

$$r_t^e = \exp\left[-40\left(\sum_e ||\hat{p}_t^e - p_t^e||^2\right)\right].$$

最后，r_{ct} 惩罚角色质心p_{ct}与参考运动 $\hat{}p_{ct}$的偏差：

$$r_t^c = \exp\left[-10\left(||\hat{p}_t^c - p_t^c||^2\right)\right].$$

6 TRAINING

我们的策略使用PPO和片段化代理目标 [Schulman等人 2017]进行训练。我们维护两个网络，一个用于策略 π_θ（$a|s,\ g$），另一个用于值函数W（$s,\ g$），参数分别为 θ和 ψ。训练以回合的形式进行，在每个回合开始时，初始状态s_0从参考运动（第6.1节）中均匀采样，并通过在每个步骤从策略中采样动作来生成滚动。每个回合模拟到固定的时间视野或直到触发终止条件（第6.2节）。一旦收集了一批数据，就从数据集中采样小批量，用于更新策略和值函数。值函数使用TD(⌋ 2017]Sutton和Barto π_θ计算的目標值进行更新。策略使用从代理目标计算出的梯度进行更新，优势At 使用GAE(⌋ Schulman等人2015b计算。请参考补充材料，了解更详细的学习算法总结。

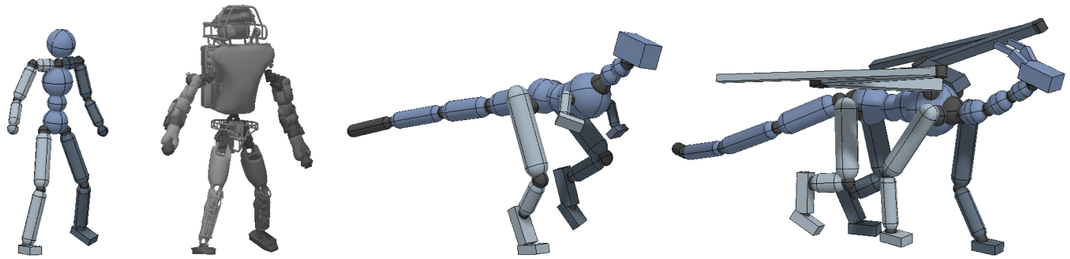
当收集了一批数据后，就从数据集中采样小批量，用于更新策略和值函数。值函数使用TD(λ) [Sutton和Barto 1998]计算的目標值进行更新。策略使用从代理目标计算出的梯度进行更新，优势At 使用GAE(λ) [Schulman等人2015b]计算。请参考补充材料，了解更详细的学习算法总结。

强化学习中一个持续存在的挑战是探索问题。由于大多数形式假设MDP未知，智能体需要利用其与环境的交互来推断MDP的结构，并发现它应该努力达到的高价值状态。为了改进探索，已经提出了许多算法改进，例如使用新颖性或信息增益的指标 [Bellemare et al. 2016; Fu et al. 2017; Houthooft et al. 2016]。然而，对训练过程中回合结构的研究较少，以及它们作为引导探索机制的潜力。在以下几节中，我们考虑两个设计决策，初始状态分布和终止条件，它们通常被视为给定强化学习问题的固定属性。我们将证明，适当的选择对于允许我们的方法学习具有挑战性的技能（如高动态踢、旋转和翻转）至关重要。使用常见的默认选择，例如固定初始状态和固定长度回合，我们发现模仿这些困难动作通常不成功。

6.1 初始状态分布

初始状态分布$p(s_0)$决定了智能体在每个回合开始时所处的状态。对于$p(s_0)$的一个常见选择是始终将智能体置于固定状态。然而，考虑模仿期望运动的任务。一个简单的策略是将角色初始化为运动的开端状态，并允许它在整个回合中向运动的结束状态前进。在这种设计中，策略必须以序列方式学习运动，首先学习运动的早期阶段，然后逐步向后期阶段推进。在掌握早期阶段之前，后期阶段几乎无法取得进展。这对于后空翻等运动可能存在问题，其中学习落地是角色从跳跃本身获得高回报的先决条件。如果策略无法成功落地，跳跃反而会导致更差的回报。固定初始状态的另一个缺点是随之而来的探索挑战。策略只有在访问过某个状态后才会获得奖励。因此，在访问到高回报状态之前，策略无法得知该状态是有利的。这两种缺点都可以通过修改初始状态分布来缓解。

对于许多强化学习任务来说，固定初始状态会更加方便，因为将智能体初始化在其他状态（例如物理机器人）中可能具有挑战性，或者难以获得更丰富的初始状态分布。然而，对于运动模仿任务，参考运动提供了一个丰富且信息量大的状态分布，可以用来在训练过程中指导智能体。在每个回合开始时，可以从参考运动中采样一个状态，并用来初始化智能体的状态。我们将这种策略称为参考状态初始化(RSI)。类似的策略之前已被用于平面双足行走 [Sharon和van de Panne 2005] 以及操作 [Nair等人2017年；Rajeswaran等人 2017]。通过从参考运动中采样初始状态，智能体在策略获得足够能力到达这些状态之前，就会遇到这些运动中的期望状态。例如，考虑学习执行后空翻的挑战。使用固定初始状态时，为了让角色发现空中完成一次完整旋转会带来高回报，它必须首先学会执行一个精心协调的跳跃。然而，为了让角色有动力执行这样的跳跃，它必须意识到跳跃会导致产生更高回报的状态。由于运动对起跳初始条件非常敏感，许多策略都会导致失败。因此，智能体不太可能遇到成功翻转的状态，也永远不会发现这样的高回报状态。使用RSI时，智能体在训练的早期阶段就会遇到这些有潜力的状态。RSI不仅可以仅通过奖励函数从参考运动中获取信息，还可以被解释为智能体通过更丰富的初始状态分布形式获取参考运动信息的额外渠道。



(a) Humanoid (b) Atlas (c) T-Rex (d) Dragon

Fig. 3. 3D simulated characters. Our framework is able to train policies for a wide range of character morphologies.

states from the reference motion, the agent encounters desirable states along the motion, even before the policy has acquired the proficiency needed to reach those states. For example, consider the challenge of learning to perform a backflip. With a fixed initial state, in order for the character to discover that performing a full rotation mid-air will result in high returns, it must first learn to perform a carefully coordinated jump. However, for the character to be motivated to perform such a jump, it must be aware that the jump will lead to states that yield higher rewards. Since the motion is highly sensitive to the initial conditions at takeoff, many strategies will result in failure. Thus the agent is unlikely to encounter states from a successful flip, and never discover such high reward states. With RSI, the agent immediately encounters such promising states during the early stages of training. Instead of accessing information from the reference motion only through the reward function, RSI can be interpreted as an additional channel through which the agent can access information from the reference motion in the form of a more informative initial state distribution.

6.2 Early Termination

For cyclic skills, the task can be modeled as an infinite horizon MDP. But during training, each episode is simulated for a finite horizon. An episode terminates either after a fixed period of time, or when certain termination conditions have been triggered. For locomotion, a common condition for early termination (ET) is the detection of a fall, characterized by the character's torso making contact with the ground [Peng et al. 2016] or certain links falling below a height threshold [Heess et al. 2016]. While these strategies are prevalent, they are often mentioned in passing and their impact on performance has not been well evaluated. In this work, we will use a similar termination condition as [Peng et al. 2016], where an episode is terminated whenever certain links, such as the torso or head, makes contact with the ground. Once early termination has been triggered, the character is left with zero reward for the remainder of the episode. This instantiation of early termination provides another means of shaping the reward function to discourage undesirable behaviors. Another advantages of early termination is that it can function as a curating mechanism that biases the data distribution in favor of samples that may be more relevant for a task. In the case of skills such as walking and flips, once the character has fallen, it can be challenging for it to recover and return to its nominal trajectory.

Table 1. Properties of the characters.

Property	Humanoid	Atlas	T-Rex	Dragon
Links	13	12	20	32
Total Mass (kg)	45	169.8	54.5	72.5
Height (m)	1.62	1.82	1.66	1.83
Degrees of Freedom	34	31	55	79
State Features	197	184	262	418
Action Parameters	36	32	64	94

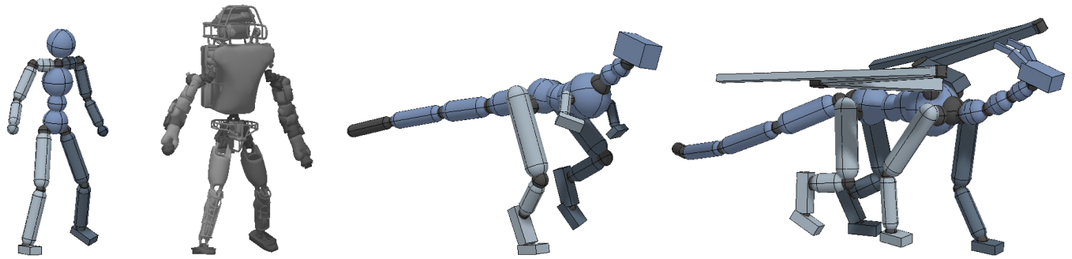
Without early termination, data collected during the early stages of training will be dominated by samples of the character struggling on the ground in vain, and much of the capacity of the network will be devoted to modeling such futile states. This phenomena is analogous to the class imbalance problem encountered by other methodologies such as supervised learning. By terminating the episode whenever such failure states are encountered, this imbalance can be mitigated.

7 MULTI-SKILL INTEGRATION

The discussion thus far has been focused on imitating individual motion clips. But the ability to compose and sequence multiple clips is vital for performing more complex tasks. In this section, we propose several methods for doing this, which are each suited for different applications. First, we need not be restricted to a single reference clip to define a desired motion style, and can instead choose to use a richer and more flexible *multi-clip* reward. Second, we can further provide the user with control over which behavior to trigger, by training a *skill selector* policy that takes in a user-specified one-hot clip-selection input. Third, we can avoid training new policies for every clip combination by instead constructing a *composite policy* out of existing single-clip policies. In this setup, multiple policies are learned independently and, at runtime, their value functions are used to determine which policy should be activated.

Multi-Clip Reward: To utilize multiple reference motion clips during training, we define a composite imitation objective calculated simply as the max over the previously introduced imitation objective applied to each of the k motion clips:

$$r_t^I = \max_{j=1,\ldots,k} r_t^j,$$



(a) 人形机器人 (b) Atlas (c) 三角龙 (d) 龙

图3. 3D模拟角色。我们的框架能够训练适用于广泛角色形态的{v1} es.

通过从参考运动中采样初始状态，智能体在策略获得足够能力到达这些状态之前，就会遇到这些运动中的期望状态。例如，考虑学习执行后空翻的挑战。使用固定初始状态时，为了让角色发现空中完成一次完整旋转会带来高回报，它必须首先学会执行一个精心协调的跳跃。然而，为了让角色有动力执行这样的跳跃，它必须意识到跳跃会导致产生更高回报的状态。由于运动对起跳初始条件非常敏感，许多策略都会导致失败。因此，智能体不太可能遇到成功翻转的状态，也永远不会发现这样的高回报状态。使用RSI时，智能体在训练的早期阶段就会遇到这些有潜力的状态。RSI不仅可以仅通过奖励函数从参考运动中获取信息，还可以被解释为智能体通过更丰富的初始状态分布形式获取参考运动信息的额外渠道。

6.2 提前终止

对于周期性技能，任务可以建模为无限视野的马尔可夫决策过程。但在训练过程中，每个回合都是针对有限视野进行模拟的。一个回合会在固定时间后终止，或者当触发某些终止条件时终止。对于运动能力，一个常见的提前终止（ET）条件是检测到跌倒，其特征是角色的躯干接触地面 [Peng 等人 2016] 或某些连接部件低于高度阈值 [Heess 等人 2016]。尽管这些策略很普遍，但它们通常只是顺便提及，且其对性能的影响尚未得到充分评估。在本工作中，我们将使用与 [Peng 等人 2016],类似的终止条件，即当某些连接部件（如躯干或头部）接触地面时终止回合。一旦触发提前终止，角色在回合剩余时间内将获得零奖励。这种提前终止的实现方式为塑造奖励函数以抑制不期望行为提供了另一种手段。提前终止的另一个优势在于，它可以作为一个筛选机制，使数据分布偏向于可能对任务更相关的样本。在行走和翻转等技能中，一旦角色跌倒，它可能难以恢复并回到正常轨迹。

表1. 角色的属性。

属性	人形机器人	Atlas	三角龙	龙
链接	13	12	20	32
总质量 (kg)	45	169.8	54.5	72.5
高度 (m)	1.62	1.82	1.66	1.83
自由度	34	31	55	79
状态特征	197	184	262	418
动作参数	36	32	64	94

如果没有提前终止，训练早期收集的数据将主要由角色徒劳挣扎于地面的样本主导，网络的大部分能力将用于建模这种徒劳的状态。这种现象类似于其他方法（如监督学习）遇到的类别不平衡问题。通过在遇到此类失败状态时终止回合，这种不平衡可以得到缓解。

7 多技能集成

迄今为止的讨论都集中在模仿单个运动片段。但组合和序列化多个片段的能力对于执行更复杂的任务至关重要。在本节中，我们提出了几种用于执行此操作的方法，每种方法都适用于不同的应用。首先，我们不必局限于单个参考片段来定义期望运动风格，而可以选择使用更丰富和更灵活的<样式 id='1'>多片段</样式>奖励。其次，我们可以通过训练一个<样式 id='3'>技能选择器</样式>策略来进一步为用户提供对触发哪种行为的控制，该策略接收用户指定的one-hot片段选择输入。第三，我们可以通过构建现有单片段策略的<样式 id='5'>复合策略</样式>来避免为每个片段组合训练新的策略。在此设置中，多个策略独立学习，在运行时，它们的价值函数用于确定应激活哪个策略。

多片段奖励：为了在训练中利用多个参考运动片段，我们定义了一个复合模仿目标，其简单地计算为先前引入的模仿目标应用于每个k运动片段的最大值：

$$r_t^I = \max_{j=1,\ldots,k} r_t^j,$$

where r_t^j is the imitation objective with respect to the jth clip. We will show that this simple composite objective is sufficient to integrate multiple clips into the learning process. Unlike [Peng et al. 2017a], which required a manually crafted kinematic planner to select a clip for each walking step, our objective provides the policy with the flexibility to select the most appropriately clip for a given situation, and the ability to switch between clips whenever appropriate, without the need to design a kinematic planner.

Skill Selector: Besides simply providing the policy with multiple clips to use as needed to accomplish a goal, we can also provide the user with control over which clip to use at any given time. In this approach, we can train a policy that simultaneously learns to imitate a set of diverse skills and, once trained, is able to execute arbitrary sequences of skills on demand. The policy is provided with a goal g_t represented by a one-hot vector, where each entry $g_{t,i}$ corresponds to the motion that should be executed. The character's goal then is to perform the motion corresponding to the nonzero entry of g_t. There is no additional task objective r_t^G, and the character is trained only to optimize the imitation objective r_t^I, which is computed based on the currently selected motion $r_t^I = r_t^i$, where $g_{t,i} = 1$ and $g_{t,j} = 0$ for $j \neq i$. During training, a random g_t is sampled at the start of each cycle. The policy is therefore required to learn to transition between all skills within the set of clips.

Composite Policy: The previously described methods both learn a single policy for a collection of clips. But requiring a network to learn multiple skills jointly can be challenging as the number of skills increases, and can result in the policy failing to learn any of the skills adequately. An alternative is to adopt a divide-and-conquer strategy, where separate policies are trained to perform different skills, and then integrated together into a composite policy. Since the value function provides an estimate of a policy's expected performance in a particular state, the value functions can be leveraged to determine the most appropriate skill to execute in a given state. Given a set of policies and their value functions $\{V^i(s), \pi^i(a|s)\}_{i=1}^k$, a composite policy $\Pi(a|s)$ can be constructed using a Boltzmann distribution

$$\Pi(a|s) = \sum_{i=1}^{k} p^i(s)\pi^i(a|s), \quad p^i(s) = \frac{\exp\left[V^i(s)/\mathcal{T}\right]}{\sum_{j=1}^{k}\exp\left[V^j(s)/\mathcal{T}\right]},$$

where $\mathcal{T}$ is a temperature parameter. Policies with larger expected values at a given state will therefore be more likely to be selected. By repeatedly sampling from the composite policy, the character is then able to perform sequences of skills from a library of diverse motions without requiring any additional training. The composite policy resembles the mixture of actor-critics experts model (MACE) proposed by [Peng et al. 2016], although it is even simpler as each sub-policy is trained independently for a specific skill.

8 CHARACTERS

Our characters include a 3D humanoid, an Atlas robot model, a T-Rex, and a dragon. Illustrations of the characters are available in Figure 3, and Table 1 details the properties of each character. All characters are modeled as articulated rigid bodies, with each link attached to its parent link via a 3 degree-of-freedom spherical joint, except for the knees and elbows, which are attached via 1 degree-of-freedom revolute joints. PD controllers are positioned at each joint, with manually specified gains that are kept constant across all tasks. Both the humanoid and Atlas share similar body structures, but their morphology (e.g., mass distribution) and actuators (e.g., PD gains and torque limits) differ significantly, with the Atlas being almost four times the mass of the humanoid. The T-Rex and dragon provide examples of learning behaviors for characters from keyframed animation when no mocap data is available, and illustrate that our method can be readily applied to non-bipedal characters. The humanoid character has a 197D state space and a 36D action space. Our most complex character, the dragon, has a 418D state space and 94D action space. Compared to standard continuous control benchmarks for RL [Brockman et al. 2016b], which typically have action spaces varying between 3D to 17D, our characters have significantly higher-dimensional action spaces.

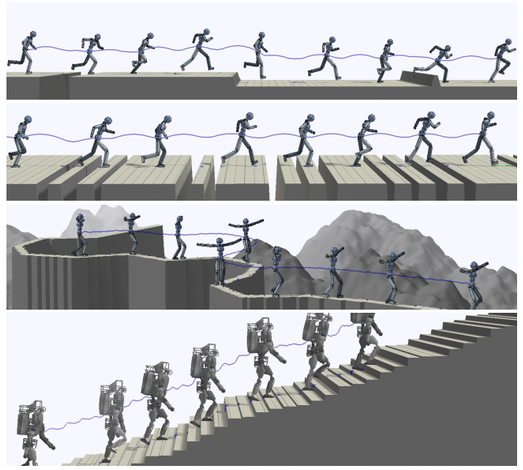


Fig. 4. Characters traversing randomly generated terrains. **Top-to-bottom:** mixed obstacles, dense gaps, winding balance beam, stairs. The blue line traces the trajectory of the character's center-of-mass.

9 TASKS

In addition to imitating a set of motion clips, the policies can be trained to perform a variety of tasks while preserving the style prescribed by the reference motions. The task-specific behaviors are encoded into the task objective r_t^G. We describe the tasks evaluated in our experiments below.

Target Heading: Steerable controllers can be trained by introducing an objective that encourages the character to travel in a target direction d_t^*, represented as a 2D unit vector in the horizontal plane. The reward for this task is given by

$$r_t^G = \exp\left[-2.5\,\max(0, v^* - v_t^T d_t^*)^2\right],$$

where v^* specifies the desired speed along the target direction d_t^*, and v_t represents the center-of-mass velocity of the simulated character. The objective therefore penalizes the character for traveling

其中 $r_{j\,t}$ 是相对于第j个片段的模仿目标。我们将证明这个简单的复合目标足以将多个片段整合到学习过程中。与需要手动设计运动规划器来为每个行走步骤选择片段的 [Peng等人2017a], 不同，我们的目标为策略提供了选择最合适片段的灵活性，以及在适当的时候切换片段的能力，而无需设计运动规划器。

技能选择器：除了简单地为策略提供多个片段以按需完成目标外，我们还可以让用户控制在任何给定时间使用哪个片段。在这种方法中，我们可以训练一个策略，该策略同时学习模仿一组多样化的技能，一旦训练完成，就能够按需执行任意技能序列。策略被提供一个由独热向量表示的目标g_t，其中每个条目 $g_{t、i}$对应于应执行的动作用。角色的目标是在 g_t 的非零条目处执行相应的动作。没有额外的任务目标r_{Gt}，角色仅被训练以优化模仿目标 r_{It}，该目标基于当前选择的动作 $r_{It} = r_{it}$计算，其中 $g_{t、i} = 1$和 $g_{t、j} = 0$对于 $j \neq i$。在训练期间，在每个周期的开始处采样一个随机的 g_t。因此，策略需要学习在片段集中的所有技能之间进行转换。

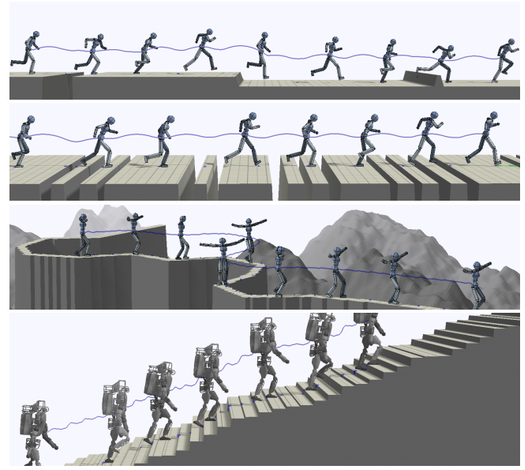


图4. 角色穿越随机生成的地形。 **从上到下：** 混合障碍、密集缝隙、弯曲平衡木、楼梯。蓝色线条追踪角色质心的轨迹。

复合策略：之前描述的方法都为一系列片段学习单个策略。但随着技能数量的增加，要求网络联合学习多个技能可能会很困难，并可能导致策略无法充分学习任何技能。另一种替代方案是采用分而治之的策略，即分别训练不同的策略来执行不同技能，然后将它们集成在一起形成复合策略。由于值函数提供了策略在特定状态下的预期性能估计，因此可以利用值函数来确定在给定状态下执行最合适的技能。给定一组策略及其值函数 $\{V_i(s),\ \pi_i(a|s)\}_{ki=1}$, 可以使用玻尔兹曼分布构建复合策略 $\Pi(a|s)$

$$\Pi(a|s) = \sum_{i=1}^{k} p^i(s)\pi^i(a|s), \quad p^i(s) = \frac{\exp\left[V^i(s)/\mathcal{T}\right]}{\sum_{j=1}^{k}\exp\left[V^j(s)/\mathcal{T}\right]},$$

其中 T 是一个温度参数。在给定状态下预期值更大的策略因此更有可能被选中。通过反复从复合策略中采样，角色就能够从包含多样化动作的库中执行技能序列，而无需任何额外的训练。复合策略类似于 [Peng 等人提出的混合演员-评论家专家模型 (MACE)，尽管它更简单，因为每个子策略都是独立地为特定技能进行训练的。

8 个角色

我们的角色包括一个3D人形机器人、一个Atlas机器人模型、一个三角龙和一个龙。角色的插图在图3中提供，表1详细说明了每个角色的属性。所有角色都被建模为关节刚体，每个连杆通过一个3自由度的球形关节连接到其父连杆，除了膝盖和肘部，它们通过1自由度的旋转关节连接。

比例-微分控制器位于每个关节处，其手动指定的增益在所有任务中保持不变。人形机器人和Atlas共享相似的身体结构，但它们的形态（例如，质量分布）和执行器（例如，PD增益和扭矩限制）差异很大，其中Atlas的质量几乎是人形机器人的四倍。三角龙和龙提供了在没有运动捕捉数据时为角色从关键帧动画学习行为的学习行为示例，并说明了我们的方法可以很容易地应用于非双足角色。人形角色具有197维状态空间和36维动作空间。我们最复杂的角色，龙，具有418维状态空间和94维动作空间。与强化学习标准连续控制基准 [Brockman等人 2016b],相比，这些基准通常具有3维到17维的动作空间，我们的角色具有显著更高维度的动作空间。

9 个任务

除了模仿一组运动片段外，策略还可以被训练在保持参考运动规定的风格的同时执行各种任务。特定任务的行为被编码到任务目标r_{Gt}中。我们将在下文描述实验中评估的任务。

目标方向： 可通过引入一个鼓励角色朝目标方向d^*_t移动的目标来训练可转向控制器，其中目标方向表示为水平平面上的二维单位向量。此任务的奖励为

$$r_t^G = \exp\left[-2.5\,\max(0, v^* - v_t^T d_t^*)^2\right],$$

其中v^*指定了沿目标方向d^*的期望速度，而$_t$ v_t表示模拟角色的质心速度。因此，目标函数会对角色在移动过程中进行惩罚

Table 2. Performance statistics of imitating various skills. All skills are performed by the humanoid unless stated otherwise. Policies are trained only to imitate a reference motion without additional task objectives. T_{cycle} is the length of the clip. $N_{samples}$ specifies the number of samples collected to train the final policy. NR represents the normalized return of the final policy averaged over 32 episodes, with 0 being the minimum possible return per episode, and 1 the maximum return. For cyclic skills, each episode has a horizon of 20s. For acyclic skills, the horizon is specified by T_{cycle}.

Skill	T_{cycle} (s)	$N_{samples}$ (10^6)	NR
Backflip	1.75	72	0.729
Balance Beam	0.73	96	0.783
Baseball Pitch	2.47	57	0.785
Cartwheel	2.72	51	0.804
Crawl	2.93	68	0.932
Dance A	1.62	67	0.863
Dance B	2.53	79	0.822
Frontflip	1.65	81	0.485
Getup Face-Down	3.28	49	0.885
Getup Face-Up	4.02	66	0.838
Headspin	1.92	112	0.640
Jog	0.80	51	0.951
Jump	1.77	86	0.947
Kick	1.53	50	0.854
Landing	2.83	66	0.590
Punch	2.13	60	0.812
Roll	2.02	81	0.735
Run	0.80	53	0.951
Sideflip	2.44	64	0.805
Spin	4.42	191	0.664
Spinkick	1.28	67	0.748
Vault 1-Handed	1.53	41	0.695
Vault 2-Handed	1.90	87	0.757
Walk	1.26	61	0.985
Atlas: Backflip	1.75	63	0.630
Atlas: Run	0.80	48	0.846
Atlas: Spinkick	1.28	66	0.477
Atlas: Walk	1.26	44	0.988
T-Rex: Walk	2.00	140	0.979
Dragon: Walk	1.50	139	0.990

slower than the desired speed along the target direction, but does not penalize it for exceeding the requested speed. The target direction is provided as the input goal $g_t = d_t^*$ to the policy. During training, the target direction is randomly varied throughout an episode. At runtime, d_t^* can be manually specified to steer the character.

Strike: In this task, the character's goal is to strike a randomly placed spherical target using specific links, such as the feet. The reward is given by

$$r_t^G = \begin{cases} 1, & \text{target has been hit} \\ \exp\left[-4||p_t^{tar} - p_t^e||^2\right], & \text{otherwise} \end{cases}$$

p_t^{tar} specifies the location of the target, and p_t^e represents the position of the link used to hit the target. The target is marked as being hit if the center of the link is within 0.2m of the target location. The goal $g_t = (p_t^{tar}, h)$ consists of the target location p_t^{tar}

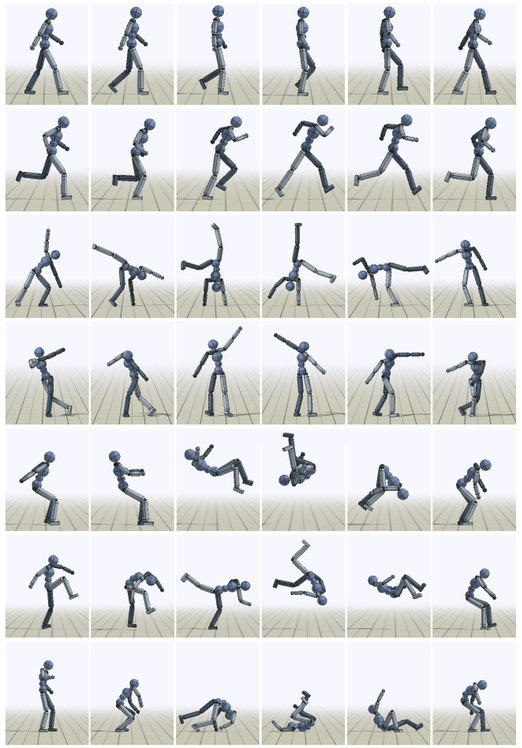


Fig. 5. Snapshots of motions from the trained policies. **Top-to-bottom:** walk, run, cartwheel, dance A, backflip, frontflip, roll.

and a binary variable h that indicates if the target has been hit in a previous timestep. As we are using feedforward networks for all policies, h acts as a memory for the state of the target. The target is randomly placed within a distance of $[0.6, 0.8]m$ from the character, the height is sampled randomly between between $[0.8, 1.25]m$, and the initial direction from the character to the target varies by $2rad$. The target location and h are reset at the start of each cycle. The memory state h can be removed by training a recurrent policy, but our simple solution avoids the complexities of training recurrent networks while still attaining good performance.

Throw: This task is a variant of the strike task, where instead of hitting a target with one of the character's links, the character is tasked with throwing a ball to the target. At the start of an episode, the ball is attached to the character's hand via a spherical joint. The joint is released at a fixed point in time during the episode. The goal g_t and reward r_t^G is the same as the strike task, but the character state s_t is augmented with the position, rotation, linear and angular velocity of the ball. The distance of the target varies between $[2.5, 3.5]m$, with height between $[1, 1.25]m$, and direction direction between $[0.7, 0.9]rad$.

表2. 模仿各种技能的性能统计。所有技能均由人形机器人执行，除非另有说明。策略仅用于模仿参考运动，没有额外的任务目标。T^{cycle}是片段的长度。$N^{samples}$指定了用于训练最终策略的样本数量。NR表示最终策略在32个回合中的平均归一化回报，其中0为每个回合的最小可能回报，1为最大回报。对于周期性技能，每个回合的视野为20s。对于非周期性技能，视野由T^{cycle}指定。

技能	T^{cycle} (s)	N样本 (10_6)	NR
后空翻	1.75	72	0.729
平衡木	0.73	96	0.783
棒球投掷	2.47	57	0.785
前滚翻	2.72	51	0.804
爬行	2.93	68	0.932
舞蹈A	1.62	67	0.863
舞蹈B	2.53	79	0.822
前空翻	1.65	81	0.485
俯卧起身	3.28	49	0.885
仰卧起身	4.02	66	0.838
头旋转	1.92	112	0.640
Jog	0.80	51	0.951
Jump	1.77	86	0.947
Kick	1.53	50	0.854
落地	2.83	66	0.590
拳击	2.13	60	0.812
Roll	2.02	81	0.735
Run	0.80	53	0.951
侧空翻	2.44	64	0.805
Spin	4.42	191	0.664
旋转踢	1.28	67	0.748
单手翻越	1.53	41	0.695
双手翻越	1.90	87	0.757
Walk	1.26	61	0.985
阿特拉斯：后空翻	1.75	63	0.630
阿特拉斯：跑步	0.80	48	0.846
阿特拉斯：旋转踢	1.28	66	0.477
Atlas: 行走	1.26	44	0.988
三角龙: 行走	2.00	140	0.979
龙: 行走	1.50	139	0.990

在目标方向上的速度低于期望速度，但不会因超出请求速度而受惩罚。目标方向作为输入目标 $g_t = d^*_t$提供给策略。在训练过程中，目标方向在整个回合中随机变化。在运行时，d^*t可以手动指定以引导角色。

击打*:* 在这个任务中，角色的目标是使用特定关节（如脚）击打随机放置的球形目标。奖励由

$$r_t^G = \begin{cases} 1, & \text{target has been hit} \\ \exp\left[-4||p_t^{tar} - p_t^e||^2\right], & \text{otherwise} \end{cases}$$

part 指定了目标的位置，*pet* 表示用于击中目标的链接的位置。如果链接的中心在目标位置0.2米范围内，则目标被标记为被击中。目标位置$gt = (part, h)$ 由目标位置$part$组成

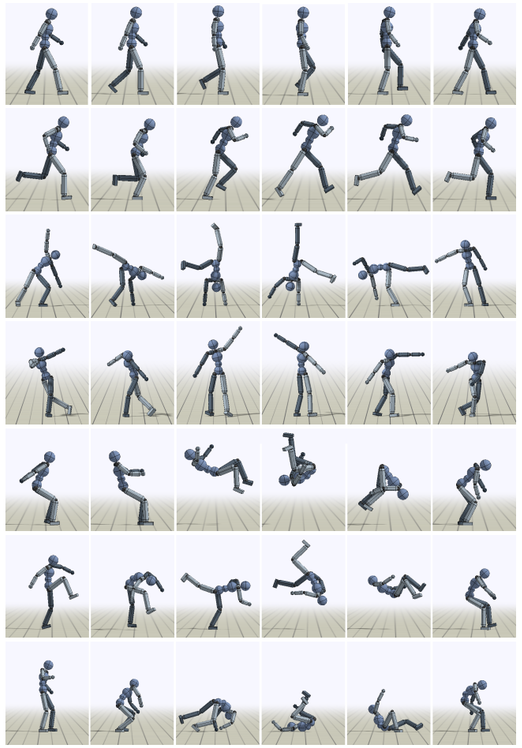
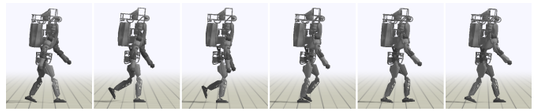


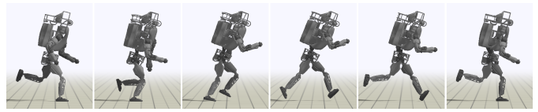
图5. 训练策略产生的运动快照。**从上到下:** 行走、跑步、前滚翻、舞蹈A、后空翻、前空翻、滚动。

和一个二进制变量 h 指示在先前时间步中目标是否被击中。由于我们为所有策略使用前馈网络，h 充当目标状态的记忆。目标在角色距离 [0.6的范围内随机放置，距离角色0.8]m，高度在 [0.8到1.25]m之间随机采样，角色到目标的初始方向变化为$2rad$。目标位置和 h 在每个周期的开始时重置。记忆状态h 可以通过训练循环策略来移除，但我们的简单解决方案避免了训练循环网络的复杂性，同时仍然达到了良好的性能。

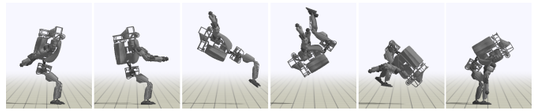
投掷： 这项任务是对击打任务的变体，其中角色不是用其连接部位击中目标，而是需要将球投向目标。在一个回合开始时，球通过球形关节连接到角色的手上。关节在回合中的固定时间点被释放。目标gt 和奖励 rGt 与击打任务相同，但角色的状态st 增加了球的位置、旋转、线速度和角速度。目标距离在 [20.5 米、3.5]米 之间变化，高度在 [1, 1.25]米 之间，方向在 [0 0.7 弧度、0.9]rad 之间。



(a) Atlas: Walk



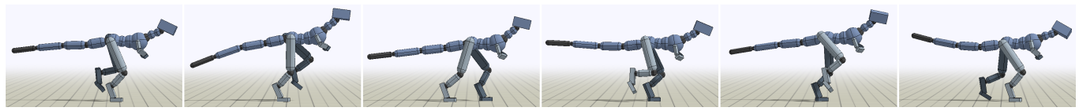
(b) Atlas: Run



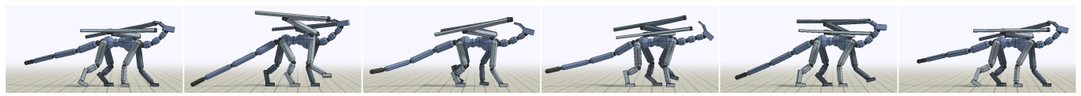
(c) Atlas: Backflip



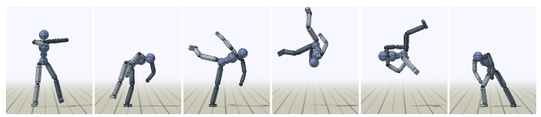
(d) Atlas: Spinkick



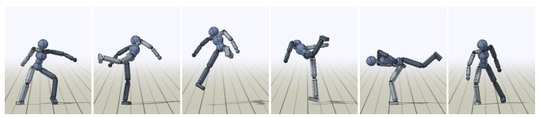
(e) T-Rex: Walk



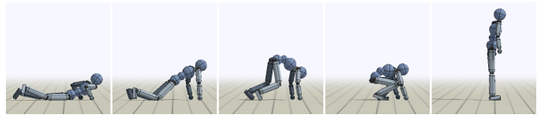
(f) Dragon: Walk



(g) Humanoid: Sideflip



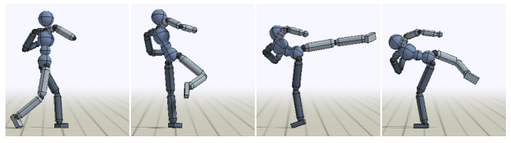
(h) Humanoid: Spin



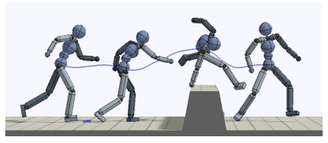
(i) Humanoid: Getup Face-Down



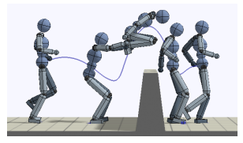
(j) Humanoid: Getup Face-Up



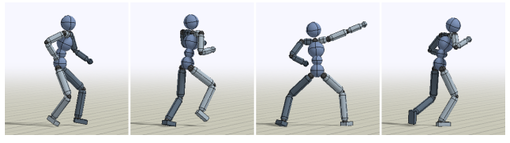
(k) Humanoid: Kick



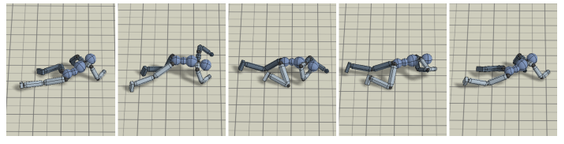
(l) Humanoid: Vault 1-Handed



(m) Humanoid: Vault 2-Handed

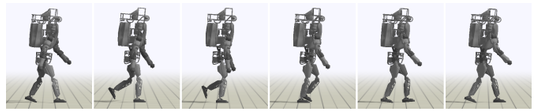


(n) Humanoid: Punch

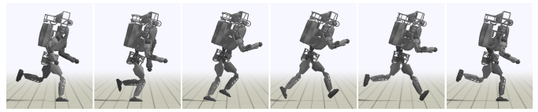


(o) Humanoid: Crawl

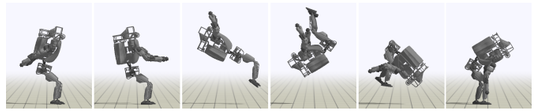
Fig. 6. Simulated characters performing various skills. Our framework is able to train policies for a broad range of characters, skills, and environments.



(a) Atlas：走路



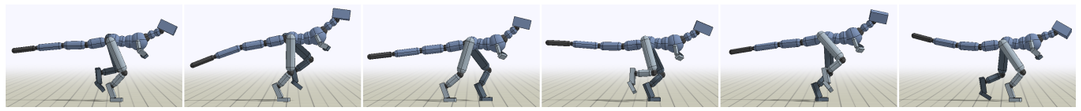
(b) Atlas：跑步



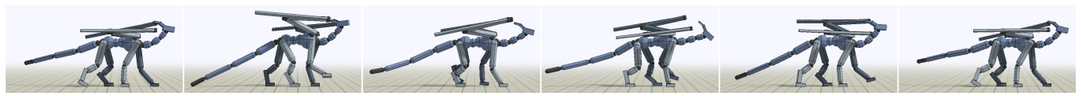
(c) Atlas：后空翻



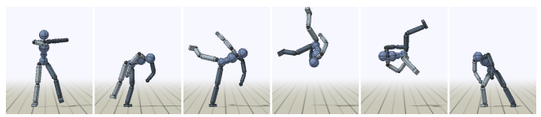
(d) Atlas：旋转踢



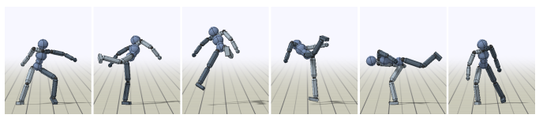
(e) 三角龙：走路



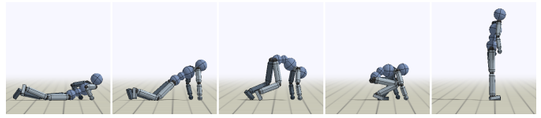
(f) 龙：行走



(g) 人形机器人：侧空翻



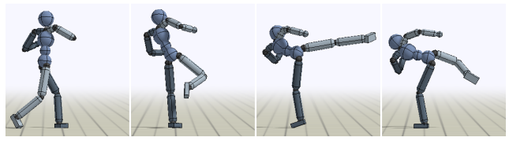
(h) 人形机器人：旋转



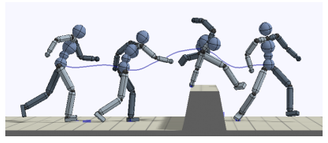
(i) 人形机器人：俯卧起身



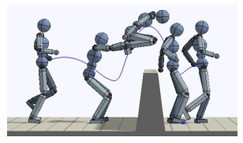
(j) 人形机器人：仰卧起身



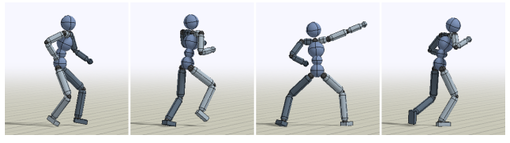
(k) 人形机器人：踢



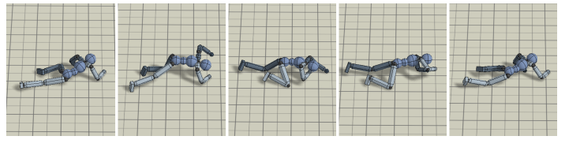
(l) 人形机器人：单手掩体1



(m) 人形机器人：双手掩体2



(n) 人形机器人：拳击



(o) 人形机器人：爬行

图6。模拟角色执行各种技能。我们的框架能够为广泛的角色、技能和环境训练策略。

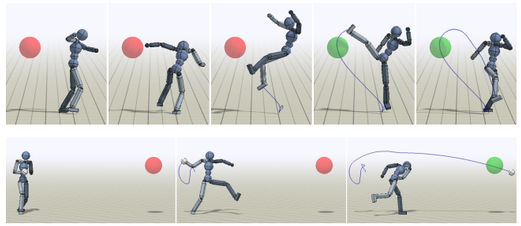


Fig. 7. **Top:** Spinkick policy trained to strike a target with the character's right foot. **Bottom:** Baseball pitch policy trained to throw a ball to a target.

Terrain Traversal: In this task, the character is trained to traverse obstacle-filled environments. The goal g_t and task-objective r_t^G are similar to those of the target heading task, except the target heading is fixed along the direction of forward progress.

We consider four environments consisting of mixed obstacles, dense gaps, a winding balance beam, and stairs. Figure 4 illustrates examples of the environments. The mixed obstacles environment is composed of gap, step, and wall obstacles similar to those presented in [Peng et al. 2016]. Each gap has a width between $[0.2, 1]m$, the height of each wall varies between $[0.25, 0.4]m$, and each step has a height between $[0.35, -0.35]m$. The obstacles are interleaved with flat stretches of terrain between $[5, 8]m$ in length. The next environment consists of sequences of densely packed gaps, where each sequence consists of 1 to 4 gaps. The gaps are $[0.1, 0.3]m$ in width, with $[0.2, 0.4]m$ of separation between adjacent gaps. Sequences of gaps are separate by $[1, 2]m$ of flat terrain. The winding balance beam environment sports a narrow winding path carved into irregular terrain. The width of the path is approximately $0.4m$. Finally, we constructed a stairs environment, where the character is to climb up irregular steps with height varying between $[0.01, 0.2]m$ and a depth of $0.28m$.

To facilitate faster training, we adopt a progressive learning approach, where the standard fully-connected networks (i.e., without the input heightmap and convolutional layers) are first trained to imitate their respective motions on flat terrain. Next, the networks are augmented with an input heightmap and corresponding convolutional layers, then trained in the irregular environments. Since the mixed obstacles and dense gaps environments follow a linear layout, the heightmaps are represented by a 1D heightfield with 100 samples spanning $10m$. In the winding balance beam environment, a 32×32 heightmap is used, covering a $3.5 \times 3.5m$ area.

10 RESULTS

The motions from the trained policies are best seen in the supplemental videos. Snapshots of the skills performed by the simulated characters are available in Figure 5, 6, and 7. The policies are executed at 30Hz. Physics simulation is performed at 1.2kHz using the Bullet physics engine [Bullet 2015]. All neural networks are built and trained using TensorFlow. The characters' motions are driven by torques computed using stable PD controllers [Tan et al. 2011]. Results for the humanoid are demonstrated for a large collection of locomotion, acrobatic, and martial arts skills, while the results for the dragon and T-Rex are demonstrated for locomotion. Each skill is learned from approximately 0.5-5s of mocap data collected from http://mocap.cs.cmu.edu and http://mocap.cs.sfu.ca. For characters such as the T-Rex and dragon, where mocap data is not available, we demonstrate that our framework is also capable of learning skills from artist-authored keyframes. Before being used for training, the clips are manually processed and retargeted to their respective characters. A comprehensive list of the learned skills and performance statistics is available in Table 2. Learning curves for all policies are available in the supplemental material. Each environment is denoted by "Character: Skill - Task". The task is left unspecified for policies that are trained solely to imitate a reference motion without additional task objectives. Performance is measured by the average return normalized by the minimum and maximum possible return per episode. Note that the maximum return may not be achievable. For example, for the throwing task, the maximum return requires moving the ball instantaneously to the target. When evaluating the performance of the policies, early termination is applied and the state of the character at the start of each episode is initialized via RSI. The weights for the imitation and task objectives are set to $\omega^I = 0.7$ and $\omega^G = 0.3$ for all tasks. More details regarding hyperparameter settings are available in the supplemental material.

Table 3. Performance statistics of imitating motion clips while also fulfilling additional task objectives.

Environment	$N_{samples}$ (10^6)	NR
Humanoid: Walk - Target Heading	85	0.911
Humanoid: Jog - Target Heading	108	0.876
Humanoid: Run - Target Heading	40	0.637
Humanoid: Spinkick - Strike	85	0.601
Humanoid: Baseball Pitch - Throw	221	0.675
Humanoid: Run - Mixed Obstacles	466	0.285
Humanoid: Run - Dense Gaps	265	0.650
Humanoid: Winding Balance Beam	124	0.439
Atlas: Walk - Stairs	174	0.808

By encouraging the policies to imitate motion capture data from human subjects, our system is able to learn policies for a rich repertoire of skills. For locomotion skills such as walking and running, our policies produce natural gaits that avoid many of the artifacts exhibited by previous deep RL methods [Merel et al. 2017; Schulman et al. 2015b]. The humanoid is able to learn a variety of acrobatic skills with long flight phases, such as backflips and spinkicks, which are are particularly challenging since the character needs to learn to coordinate its motion in mid-air. The system is also able to reproduce contact-rich motions, such as crawling and rolling, as well as motions that require coordinated interaction with the environment, such as the vaulting skills shown in Figure 6. The learned policies are robust to significant external perturbation and generate plausible recovery behaviors. The policies trained for the T-Rex and dragon demonstrate that the system can also learn from artist generated keyframes when mocap is not available and scale to much more complex characters than those that have been demonstrated by previous work using deep RL.

10.1 Tasks

In addition to imitating reference motions, the policies can also adapt the motions as needed to satisfy additional task objectives,

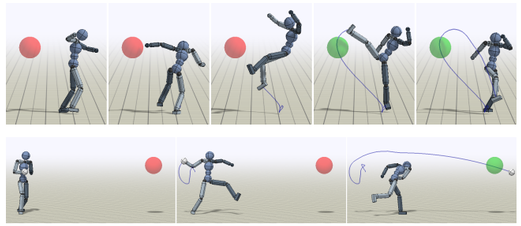


图7。**顶部：**训练角色用右脚击中目标的旋转踢策略。**底部：**训练将球投掷到目标的棒球投球策略。

地形穿越：在这个任务中，角色被训练穿越障碍环境。目标和任务目标与目标标题task相似，但目标标题固定在向前推进的方向上。

我们考虑了四个由混合障碍、密集缝隙、弯曲平衡木和楼梯组成的环境。图4说明了这些环境的示例。混合障碍环境由类似于 [Peng等人 2016]中提出的间隙、台阶和墙壁障碍物组成。每个间隙的宽度在 [0.2, 1]米之间，每个墙壁的高度在 [0.25、0.4]米之间，每个台阶的高度在 [0.35、-0.35]米之间。障碍物与长度为 [5, 8]米的平坦地形交替出现。下一个环境由密集排列的间隙序列组成，其中每个序列由1到4个间隙组成。间隙的宽度为 [0.1、0.3]米，相邻间隙之间有[0.2、0.4]米的间隔。间隙序列之间由 [1, 2]米的平坦地形隔开。弯曲平衡木环境在不规则地形上刻有一条狭窄的弯曲路径。路径宽度约为0.4米。最后，我们构建了一个楼梯环境，角色需要爬上高度在 [0.01、0.2]米之间、深度为0.28米的不规则台阶。

为了加快训练速度，我们采用渐进式学习方法，首先使用标准全连接网络（即不包含输入高度图和卷积层）在平坦地形上模仿其各自的运动。接下来，为网络添加输入高度图和相应的卷积层，然后在不规则环境中进行训练。由于混合障碍和密集缝隙环境采用线性布局，高度图表示为一维高度场，包含100个样本，跨越10米<样式 id='1'>m</样式>。在弯曲平衡木环境中，使用 32×32 高度图，覆盖3.5×3.5<样式 id='5'>m</样式>的区域。

10 结果

训练好的策略的运动效果在补充视频中最为明显。模拟角色执行技能的快照可在图5、6和7中找到。策略以30Hz的频率执行。物理模拟使用Bullet物理引擎 [Bullet 2015]以1.2kHz进行。所有神经网络都使用TensorFlow构建和训练。角色的运动由稳定PD控制器 [Tan等人 2011]计算出的扭矩驱动。人形机器人的结果展示了一系列运动、杂技和武术技能，而龙和三角龙的结果仅展示运动技能。每个技能

表3. 模仿运动片段并满足附加任务目标时的性能统计。

环境	<样式 id='1'>N</样式><样式 id='2'>样本</样式>(<样式 id='3'>10^6</样式>)	NR
人形机器人：行走 - 目标方向	85	0.911
人形机器人：慢跑 - 目标方向	108	0.876
人形机器人：跑步 - 目标方向	40	0.637
人形机器人：旋转踢 - 击球	85	0.601
人形机器人：棒球投掷 - 投掷	221	0.675
人形机器人：跑步 - 混合障碍	466	0.285
人形机器人：跑步 - 密集缝隙	265	0.650
人形机器人：弯曲平衡木	124	0.439
Atlas：行走 - 楼梯	174	0.808

是从http://mocap.cs.cmu.edu和http://mocap.cs.sfu.ca收集的约0.5-5秒的运动捕捉数据中学习的。对于T-Rex和龙等没有运动捕捉数据的角色，我们展示了我们的框架也能从艺术家创作的关键帧中学习技能。在用于训练之前，片段会经过手动处理并重新映射到相应的角色上。学习技能的完整列表和性能统计数据可在表2中找到。所有策略的学习曲线在补充材料中提供。每个环境用“角色：技能-任务”表示。对于仅用于模仿参考运动且没有额外任务目标的策略，任务未指定。性能通过每回合最小和最大可能回报归一化的平均回报来衡量。请注意，最大回报可能无法实现。例如，对于投掷任务，最大回报需要将球瞬间移动到目标位置。在评估策略性能时，会应用提前终止，并通过RSI初始化每个回合中角色的状态。模仿和任务目标的权重对所有任务分别设置为 $\omega I = 0.7$和 $\omega G = 0.3$。有关超参数设置的更多细节可在补充材料中找到。

通过鼓励策略模仿人类受试者的运动捕捉数据，我们的系统能够学习丰富的技能库。对于行走和跑步等运动技能，我们的策略产生了自然步态，避免了先前深度强化学习方法表现出的许多伪影 [Merel等人2017年；Schulman等人2015b]。人形机器人能够学习具有长飞行阶段的杂技技能，如后空翻和旋转踢，这些技能尤其具有挑战性，因为角色需要在空中协调其运动。该系统还能够再现接触丰富的动作，如爬行和滚动，以及需要与环境协调交互的动作，如图6中所示的跳跃技能。学习到的策略对显著外部扰动具有鲁棒性，并能生成合理的恢复行为。为三角龙和龙训练的策略表明，该系统也可以在运动捕捉不可用时从艺术家生成的关键帧中学习，并扩展到比先前使用深度强化学习方法演示的更复杂的角色。

10.1 任务

除了模仿参考运动，策略还可以根据需要调整运动以满足额外的任务目标，

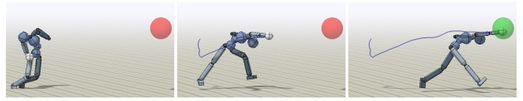


Fig. 8. Policy trained for the throw task without a reference motion. Instead of throwing the ball, the character learns to run towards the target.

Table 4. Success rate of policies trained with the imitation or task objectives disabled. Each policy is evaluated over 100 trials. Simply imitating the reference motions proves insufficient for fulfilling the task objectives. Training without a reference motion produces policies that develop awkward, but functional, strategies for satisfying the task objectives.

Environment	$r^I + r^G$	r^I	r^G
Humanoid: Strike - Spinkick	99%	19%	55%
Humanoid: Baseball Pitch - Throw	75%	5%	93%

such as following a target heading and throwing a ball to a randomly placed target. Performance statistics for each task are available in Table 3. To investigate the extent to which the motions are adapted for a particular task, we compared the performance of policies trained to optimize both the imitation objective r^I and the task objective r^G to policies trained only with the imitation objective. Table 4 summarizes the success rates of the different policies. For the throwing task, the policy trained with both objectives is able to hit the target with a success rate of 75%, while the policy trained only to imitate the baseball pitch motion is successful only in 5% of the trials. Similarly, for the strike task, the policy trained with both objectives successfully hits 99% of the targets, while the policy trained only to imitate the reference motion has a success rate of 19%. These results suggest that simply imitating the reference motions is not sufficient to successfully perform the tasks. The policies trained with the task objective are able to deviate from the original reference motion and developing additional strategies to satisfy their respective goals. We further trained policies to optimize only the task objective, without imitating a reference motion. The resulting policies are able to fulfill the task objectives, but without a reference motion, the policies develop unnatural behaviors, similar to those produced by prior deep RL methods. For the throw task, instead of throwing the ball, the policy adopts an awkward but functional strategy of running towards the target with the ball. Figure 8 illustrates this behavior.

10.2 Multi-Skill Integration

Our method can construct policies for reproducing a wide range of individual motion clips. However, many more complex skills require choosing from among a set of potential behaviors. In this section, we discuss several methods by which our approach can be extended to combine multiple clips into a single compound skill.

Multi-Clip Reward: To evaluate the multi-clip reward, we constructed an imitation objective from 5 different walking and turning clips. Using this reward, a humanoid policy is then trained to walk while following a desired heading. The resulting policy learns to utilize a variety of agile stepping behaviours in order to follow the target heading. To determine if the policy is indeed learning to imitate multiple clips, we recorded the ID of the clip that best matches the motion of the simulated character at each timestep as it follows

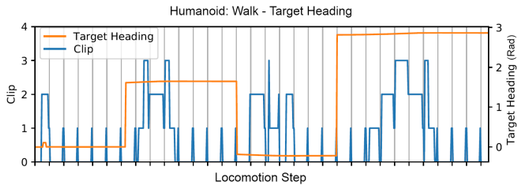


Fig. 9. Visualization of the walking reference motion clip that most closely matches the simulated character's motion as the character follows a changing target heading. Clip 0 is a forward walking motion, and clip 1-4 are turning motions. The policy is trained with the multi-clip reward.

a changing target heading. The best matching clip is designated as the clip that yields the maximum reward at the current timestep. Figure 9 records the best matching clips and the target heading over a 20*s* episode. Clip 0 corresponds to a forward walking motion, and clips 1-4 are different turning motions. When the target heading is constant, the character's motion primarily matches the forward walk. When the target heading changes, the character's motion becomes more closely matched with the turning motions. After the character has realigned with the target heading, the forward walking clip once again becomes the best match. This suggests that the multi-clip reward function does indeed enable the character to learn from multiple clips of different walking motions. When using the multi-clip reward, we found that the clips should generally be from a similar type of motion (e.g. different kinds of walks and turns). Mixing very different motions, like a sideflip and a frontflip together, can result in the policy imitating only a subset of the clips. For more diverse clips, we found the composite policy to be a more effective integration method.

Skill Selector: Using the one-hot vector representation, we trained a policy to perform various flips, and another to jump in different directions. The flip policy is trained to perform a frontflip, backflip, left sideflip, and right sideflip. The jump policy is trained to perform a forward, backward, left, and right jump. The first and last frame of each clip are duplicated to ensure that all clips within each set have the same cycle period. Once trained, the policies are able to perform arbitrary sequences of skills from their respective repertoires. The one-hot vector also provides a convenient interface for users to direct the character in real-time. Footage of the humanoid executing sequences of skills specified by a user is available in the supplemental video.

Composite Policy: While the multi-clip objective can be effective for integrating multiple clips that correspond to the same category (e.g., walking), we found it to be less effective for integrating clips of more diverse skills. To integrate a more diverse corpus of skills, we constructed a composite policy from the backflip, frontflip, sideflip, cartwheel, spinkick, roll, getup face-down, and getup face-up policies. The output of the value functions are normalized to be between [-1, 1] and the temperature is set to $\mathcal{T} = 0.3$. A new skill is sampled from the composite policy at the start of each cycle, and the selected skill is executed for a full cycle before selecting a new skill. To prevent the character from repeatedly executing the same skill, the policy is restricted to never sample the same skill in consecutive

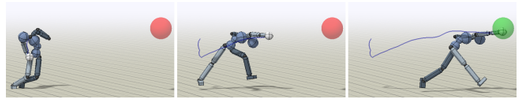


图8。为投掷任务训练的策略，没有参考运动。角色没有投掷球，而是学习跑向目标。

表4。禁用模仿或任务目标训练的策略的成功率。每个策略在100次试验中评估。仅模仿参考运动不足以实现任务目标。没有参考运动时，训练产生的策略会发展出笨拙但功能性的策略来满足任务目标。

环境	$r^I + r^G$	r^I	r^G
人形机器人：击打 - 旋转踢	99%	19%	55%
人形机器人：棒球投掷 - 投掷	75%	5%	93%

例如跟随目标方向并将球投掷到随机放置的目标处。每个任务的性能统计数据在表3中提供。为了研究运动为特定任务调整的程度，我们将优化模仿目标 rI 和任务目标 rG 的策略的性能与仅使用模仿目标的策略的性能进行了比较。表4总结了不同策略的成功率。对于投掷任务，同时使用两个目标的策略能够以75%的成功率击中目标，而仅模仿棒球投掷运动的策略仅在5%的试验中成功。同样，对于击球任务，同时使用两个目标的策略成功击中99%的目标，而仅模仿参考运动的策略成功率为19%。这些结果表明，仅模仿参考运动不足以成功执行任务。使用任务目标的策略能够偏离原始参考运动并制定额外的策略以满足其各自目标。我们进一步训练了仅优化任务目标的策略，而不模仿参考运动。生成的策略能够完成任务目标，但由于没有参考运动，策略会发展出不自然的动作，类似于先前深度强化学习方法产生的动作。对于投掷任务，策略不是投掷球，而是采取一种笨拙但功能性的策略——带着球跑向目标。图8说明了这种行为。

10.2 多技能集成

我们的方法可以构建策略以复现各种单个运动片段。然而，许多更复杂的技能需要从一组潜在行为中进行选择。在本节中，我们讨论了我们的方法如何通过几种方法扩展，将多个片段组合成一个复合技能。

多片段奖励：为了评估多片段奖励，我们构建了一个模仿目标，该目标由5个不同的行走和转向片段组成。使用这个奖励，然后训练一个人形策略，使其在遵循期望航向的同时行走。由此产生的策略学习利用各种敏捷的步态行为来遵循目标航向。为了确定策略是否确实在学习模仿多个片段，我们在模拟角色遵循时，记录了在每个时间步长上与模拟角色的运动最匹配的片段的ID

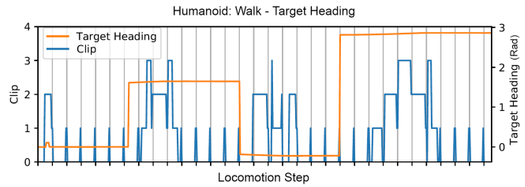


图9. 当角色跟随变化的目标方向时，与模拟角色运动最匹配的行走参考运动片段的可视化。片段0是一个向前行走运动，片段1-4是转向运动。策略使用多片段奖励进行训练。

一个变化的目標方向。最佳匹配的片段被指定为在当前时间步产生最大奖励的片段。图9记录了在20秒的片段中最佳匹配的片段和目標方向。片段0对应于向前行走动作，而片段1-4是不同的转向动作。当目標方向保持不变时，角色的动作主要匹配向前行走。当目標方向变化时，角色的动作变得更紧密地匹配转向动作。在角色与目標方向重新对齐后，向前行走的片段再次成为最佳匹配。这表明多片段奖励函数确实使角色能够从不同行走动作的多个片段中学习。在使用多片段奖励时，我们发现片段通常应来自相似类型的动作（例如，不同种类的行走和转向）。混合非常不同的动作，如侧空翻和前空翻，会导致策略仅模仿片段的子集。对于更多样化的片段，我们发现复合策略是更有效的集成方法。

技能选择器：使用独热向量表示，我们训练了一个策略执行各种翻转，以及另一个在不同方向跳跃。翻转策略被训练执行前空翻、后空翻、左侧空翻和右侧空翻。跳跃策略被训练执行向前跳跃、向后跳跃、向左跳跃和向右跳跃。每个片段的第一帧和最后一帧被复制，以确保每个集合内的所有片段具有相同的周期。训练完成后，策略能够从它们各自的技能库中执行任意技能序列。独热向量还为用户提供了一个方便的接口，用于实时控制角色。人形机器人执行用户指定的技能序列的片段可在补充视频中找到。

复合策略：虽然多片段目标对于整合属于同一类别的多个片段（例如行走）很有效，但我们发现它对于整合技能更多样化的片段效果较差。为了整合更多样化的技能语料库，我们从后空翻、前空翻、侧空翻、前滚翻、旋转踢、滚动、俯卧起身和仰卧起身策略构建了一个复合策略。值函数的输出被归一化到 [-1, 1]之间，温度设置为T = 0。3。在每个周期的开始，从复合策略中采样一个新技能，并在选择新技能之前执行所选技能一个完整周期。为了防止角色重复执行同一技能，策略被限制在连续周期中从不采样同一技能。

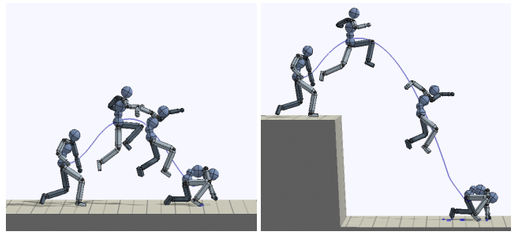


Fig. 10. **Left:** Original landing motion on flat terrain. **Right:** Policy trained to imitating landing motion while jumping down from a $2m$ ledge. Despite being provided only with a motion recorded on flat terrain, the policy is able to adapt the skill to jump down from a tall ledge.

cycles. Unlike the skill selector policies, the individual policies in the composite policy are never explicitly trained to transition between different skills. By using the value functions to guide the selection of skills, the character is nonetheless able to successfully transition between the various skills. When the character falls, the composite policy activates the appropriate getup policy without requiring any manual scripting, as shown in the supplemental video.

10.3 Retargeting

Due to modeling discrepancies between simulation and the real world, the dynamics under which a motion capture clip was recorded can differ dramatically from the dynamics of the simulated environments. Furthermore, keyframed motions may not be physically correct at all. To evaluate our framework's robustness to these discrepancies, we trained policies to perform similar skills with different character models, environments, and physics.

Character Retargeting: To demonstrate the system's capabilities in retargeting motions to different characters, we trained policies for walking, running, backflips and spinkicks on a simulated model of the Atlas robot from http://www.mujoco.org/forum/index.php?resources/atlas-v5.16/. The Atlas has a total mass of 169.8kg, almost four times the mass of the humanoid, as well as a different mass distribution. The serial 1D revolute joints in the original Atlas model are aggregated into 3D spherical joints to facilitate simpler retargeting of the reference motions. To retarget the motion clips, we simply copied the local joint rotations from the humanoid to the Atlas, without any further modification. New policies are then trained for the Atlas to imitate the retargeted clips. Despite the starkly different character morphologies, our system is able to train policies that successfully reproduce the various skills with the Atlas model. Performance statistics of the Atlas policies are available in Table 2. The performance achieved by the Atlas policies are comparable to those achieved by the humanoid. Qualitatively, the motions generated by the Atlas character closely resemble the reference clips. To further highlight the differences in the dynamics of the characters, we evaluated the performance of directly applying policies trained for the humanoid to the Atlas. The humanoid policies, when applied to the Atlas, fail to reproduce any of the skills, achieving a normalized return of 0.013 and 0.014 for the run and backflip, compared to 0.846

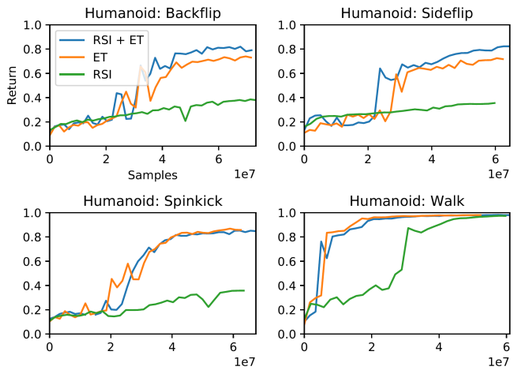


Fig. 11. Learning curves for policies trained with and without reference state initialization (RSI) and early termination (ET).

Table 5. Normalized average returns of policies trained with and without reference state initialization (RSI) and early termination (ET).

Skill	RSI + ET	ET	RSI
Backflip	0.791	0.730	0.379
Sideflip	0.823	0.717	0.355
Spinkick	0.848	0.858	0.358
Walk	0.980	0.981	0.974

and 0.630 achieved by policies that were trained specifically for the Atlas but using the same reference clips.

Environment Retargeting: While most of the reference motions were recorded on flat terrain, we show that the policies can be trained to adapt the motions to irregular environments. First, we consider the case of retargeting a landing motion, where the original reference motion is of a human subject jumping and landing on flat ground. From this reference motion, we trained a character to reproduce the motion while jumping down from a $2m$ ledge. Figure 10 illustrates the motion from the final policy. The system was able to adapt the reference motion to a new environment that is significantly different from that of the original clip.

Next, we explore vision-based locomotion in more complex procedurally generated environments. By augmenting the networks with a heightmap input, we are able to train the humanoid to run across terrains consisting of random obstacles. Examples of the environments are available in Figure 4. Over the course of training, the policies are able to adapt a single clip of a forward running motion into a variety of strategies for traversing across the different classes of obstacles. Furthermore, by training a policy to imitate a balance beam walk, the character learns to follow a narrow winding path using only a heightmap for pathfinding. The balance beam policy was trained with only a forward walking clip, but is nonetheless able to develop turning motions to follow the winding path. In addition to the humanoid, we also trained a policy for the Atlas to climb up stairs with irregular step heights. The policy was able to adapt the original walking clip on flat terrain to climb the steps, although the resulting motion still exhibits an awkward gait. We suspect that

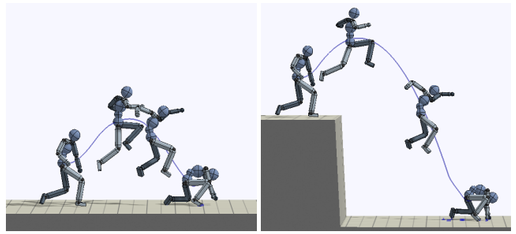


图 10. **左：** 平坦地形上的原始落地动作。 **右：** 训练策略以模仿从 2米高台跳跃落地的动作。尽管仅提供了平坦地形上的动作记录，该策略仍能将技能适应于从高台跳跃落地。

与技能选择器策略不同，复合策略中的单个策略从未被明确训练以在不同技能之间进行转换。通过使用值函数来指导技能的选择，角色仍然能够成功地在不同技能之间进行转换。当角色摔倒时，复合策略会激活适当的起身策略，而无需任何手动脚本，如补充视频所示。

10.3 重定向

由于模拟与现实世界之间存在建模差异，运动捕捉片段录制时的动力学可能与模拟环境的动力学差异很大。此外，关键帧动画可能完全不符合物理规律。为了评估我们的框架对这些差异的鲁棒性，我们使用不同的角色模型、环境和物理训练策略来执行类似的技能。

角色重定向：为了展示系统将运动重定向到不同角色的能力，我们在 http://www.mujoco.org/forum/index.php?resources/atlas-v5.16/ 提供的Atlas机器人的模拟模型上训练了用于行走、跑步、后空翻和旋转踢的策略。Atlas的总质量为169.8kg，几乎是人形机器人的四倍，并且质量分布也不同。原始Atlas模型中的串行1D旋转关节被聚合成3D球形关节，以简化参考运动的重定向。为了重定向运动片段，我们简单地将从人形机器人复制到Atlas的局部关节旋转，没有任何进一步修改。然后为Atlas训练新的策略来模仿重定向的片段。尽管角色的形态差异很大，我们的系统能够训练策略，成功地使用Atlas模型再现各种技能。Atlas策略的性能统计数据在表2中。Atlas策略达到的性能与人形机器人达到的性能相当。从定性的角度来看，Atlas角色生成的运动与人形机器人参考片段非常相似。为了进一步突出角色动力学的差异，我们评估了将为人形机器人训练的策略直接应用于Atlas的性能。当将人形机器人策略应用于Atlas时，它们无法再现任何技能，跑步和后空翻的归一化回报分别为0.013和0.014，而人形机器人策略为0.846

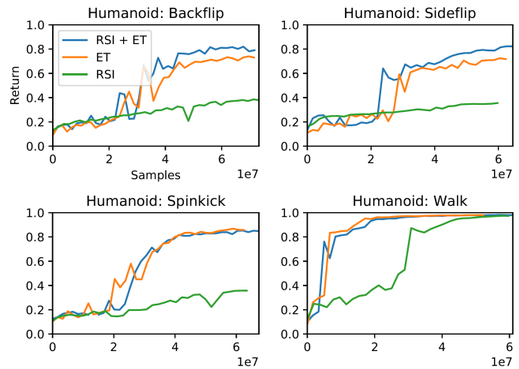


图 11. 使用参考状态初始化（RSI）和提前终止（ET）以及不使用参考状态初始化（RSI）和提前终止（ET）训练的策略的学习曲线

表 5. 使用参考状态初始化（RSI）和提前终止（ET）训练的策略的归一化平均回报。

技能	RSI + ET	ET	RSI
后空翻	0.791	0.730	0.379
侧空翻	0.823	0.717	0.355
旋转踢	0.848	0.858	0.358
Walk	0.980	0.981	0.974

并且，使用专门为 Atlas 训练但使用相同参考片段的策略达到了0.630。

环境重定向：虽然大多数参考运动是在平坦地形上录制的，但我们展示了策略可以被训练以适应不规则环境。首先，我们考虑重定向落地运动的情况，其中原始参考运动是一个人类主体在平坦地面上跳跃和落地。从这个参考运动中，我们训练一个角色在从一个 2米台阶跳下时重现该运动。图 10 说明了最终策略的运动。该系统能够将参考运动适应到一个与原始片段差异显著的新环境。

接下来，我们探索了在更复杂的程序生成环境中基于视觉的运动。通过给网络增加高度图输入，我们能够训练人形机器人跨越由随机障碍物组成的地形进行跑步。这些环境示例可以在图4中找到。在训练过程中，策略能够将一个向前跑步的片段适应为跨越不同类别障碍物的各种策略。此外，通过训练策略模仿平衡木行走，角色学习到仅使用高度图进行路径规划来跟随狭窄蜿蜒的路径。平衡木策略仅使用一个向前行走的片段进行训练，但仍然能够发展出转向动作来跟随蜿蜒路径。除了人形机器人，我们还训练了Atlas的策略，使其能够攀爬步高不规则的楼梯。策略能够将平坦地形上的原始行走片段适应为攀爬台阶，尽管产生的动作仍然显得笨拙。我们推测，

the problem is partly related to the walking reference motion being ill-suited for the stairs environment.

Physics Retargeting: To further evaluate the framework's robustness to discrepancies between the dynamics of the motion capture data and simulation, we trained policies to perform a spinkick and cartwheel under moon gravity ($1.622m/s^2$). Despite the difference in gravity, the policies were able to adapt the motions to the new dynamics, achieving a return of 0.792 for the spinkick and 0.688 for the cartwheel.

10.4 Ablations

To evaluate the impact of our design decisions, we compare our full method against alternative training schemes that disable some of the components. We found that the reference state initialization and early termination are two of the most important components of our training procedure. The comparisons include training with reference state initialization and with a fixed initial state, as well as training with early termination and without early termination. Without early termination, each episode is simulated for the full 20s. Figure 11 compares the learning curves with the different configurations and Table 5 summarizes the performance of the final policies. During evaluation, early termination is disabled and the character is initialized to a fixed state at the start of each episode. Due to the time needed to train each policy, the majority of performance statistics are collected from one run of the training process. However, we have observed that the behaviors are consistent across multiple runs. Early termination proves to be crucial for reproducing many of the skills. By heavily penalizing the character for making undesirable contacts with the ground, early termination helps to eliminate local optima, such as those where the character falls and mimes the motions as it lies on the ground. RSI also appears vital for more dynamic skills that have significant flight phases, such as the backflip. While the policies trained without RSI appear to achieve a similar return to those trained with RSI, an inspection of the resulting motions show that, without RSI, the character often fails to reproduce the desired behaviours. For the backflip, without RSI, the policy never learns to perform a full mid-air flip. Instead, it performs a small backwards hop while remaining upright.

10.5 Robustness

To determine the policies' robustness to external perturbations, we subjected the trained policies to perturbation forces and recorded the maximum force the character can tolerate before falling. The perturbation forces are applied halfway through a motion cycle to the character's pelvis for 0.2s. The magnitude of the force is increased by 10N until the character falls. This procedure is repeated for forces applied along the forward direction (x-axis) and sideway direction (z-axis). Table 6 summarizes the results from the experiments on the humanoid. The learned policies show comparable-or-better robustness than figures reported for SAMCON [Liu et al. 2016]. The run policy is able to recover from $720N \times 0.2s$ forward pushes, while the spin-kick policy is able to survive $600N \times 0.2s$ perturbations in both directions. Note that no external perturbations are applied during the training process; we suspect that the policies' robustness is in large part due to the exploration noise applied by the stochastic policy used during training.

Table 6. Maximum forwards and sideways push each policy can tolerate before falling. Each push is applied to the character's pelvis for 0.2s.

Skill	Forward (N)	Sideway (N)
Backflip	440	100
Cartwheel	200	470
Run	720	300
Spinkick	690	600
Walk	240	300

11 DISCUSSION AND LIMITATIONS

We presented a data-driven deep reinforcement learning framework for training control policies for simulated characters. We show that our method can produce a broad range of challenging skills. The resulting policies are highly robust and produce natural motions that are nearly indistinguishable from the original motion capture data in the absence of perturbations. Our framework is able to retarget skills to a variety of characters, environments, and tasks, and multiple policies can be combined into composite policies capable of executing multiple skills.

Although our experiments illustrate the flexibility of this approach, there are still numerous limitations to be addressed in future work. First, our policies require a phase variable to be synchronized with the reference motion, which advances linearly with time. This limits the ability of the policy to adjust the timing of the motion, and lifting this limitation could produce more natural and flexible perturbation recoveries. Our multi-clip integration approach works well for small numbers of clips, but has not yet been demonstrated on large motion libraries. The PD controllers used as the low-level servos for the characters still require some insight to set properly for each individual character morphology. The learning process itself is also quite time consuming, often requiring several days per skill, and is performed independently for each policy. Although we use the same imitation reward across all motions, this is still currently based on a manually defined state-similarity metric. The weighting of the reward terms also needs to be defined with some care.

We believe this work nevertheless opens many exciting directions for exploration. In future work, we wish to understand how the policies might be deployed on robotic systems, as applied to locomotion, dexterous manipulation, and other tasks. It would be interesting to understand the learned control strategies and compare them to the equivalent human strategies. We wish to integrate diverse skills that would enable a character to perform more challenging tasks and more complex interactions with their environments. Incorporating hierarchical structure is likely to be beneficial towards this goal.

ACKNOWLEDGEMENTS

We thank KangKang Yin, Libin Liu, and Ziva Dynamics for providing reference motion data for this project, the anonymous reviewers for their helpful feedback, and AWS for providing computational resources. This research was funded by an NSERC Postgraduate Scholarship, a Berkeley Fellowship for Graduate Study, and ONR PECASE N000141612723.

REFERENCES

Shailen Agrawal, Shuo Shen, and Michiel van de Panne. 2013. Diverse Motion Variations for Physics-based Character Animation. *Symposium on Computer Animation* (2013).

这个问题部分与行走参考运动不适合楼梯环境有关。

物理重定向：为了进一步评估该框架对动作捕捉数据动力学与模拟之间差异的鲁棒性，我们在月球重力（$1.622m/s^2$）下训练了策略以执行旋转踢和前滚翻。尽管重力不同，策略能够适应新的动力学，旋转踢的回报为0.792，前滚翻的回报为0.688。

10.4 消融实验

为了评估我们的设计决策的影响，我们将完整方法与禁用部分组件的替代训练方案进行比较。我们发现参考状态初始化和提前终止是我们训练过程中最重要的两个组件。比较包括使用参考状态初始化和使用固定初始状态进行训练，以及使用提前终止和不使用提前终止进行训练。在不使用提前终止的情况下，每个回合模拟时间为完整的 20秒。图 11 比较了不同配置的学习曲线，表 5 总结了最终策略的性能。在评估期间，禁用提前终止，角色在每个回合开始时初始化为固定状态。由于训练每个策略所需的时间，大多数性能统计数据是从训练过程的一次运行中收集的。然而，我们观察到行为在多次运行中是一致的。提前终止被证明对于复制许多技能至关重要。通过严厉惩罚角色与地面进行不希望的接触，提前终止有助于消除局部最优解，例如角色摔倒并在地上模仿动作的情况。RSI 也对具有显著飞行阶段（如后空翻）的更动态的技能至关重要。虽然不使用 RSI 训练的策略似乎实现了与使用 RSI 训练的策略相似的回报，但检查结果动作表明，在不使用 RSI 的情况下，角色通常无法复制所需行为。对于后空翻，在不使用 RSI 的情况下，策略永远不会学会执行完整的空中翻转。相反，它执行一个小小的向后跳跃，同时保持直立。

10.5 稳定性

为确定策略对外部扰动的鲁棒性，我们将训练好的策略暴露于扰动力下，并记录角色在跌倒前能承受的最大力。扰动力在运动周期中点施加于角色的骨盆，持续0.2秒。力的幅度以10牛的步长增加，直至角色跌倒。该步骤分别沿向前方向（x轴）和侧向方向（z轴）重复进行。表6总结了人形机器人实验的结果。学习到的策略表现出与SAMCON [Liu et al. 2016]报告的鲁棒性相当或更好的性能。跑步策略能够从720牛 ×0.2秒的向前推力中恢复，而旋转踢策略能够在两个方向上承受600牛 ×0.2秒的扰动。请注意，在训练过程中未施加任何外部扰动；我们推测策略的鲁棒性在很大程度上归因于训练期间随机策略施加的探索噪声。

表 6. 每个策略在跌倒前能承受的最大向前和侧向推力。每次推力都施加到角色的骨盆上，持续 0.2秒。

技能	向前 (N)	侧向 (N)
后空翻	440	100
前滚翻	200	470
Run	720	300
旋转踢	690	600
Walk	240	300

11 讨论与局限性

我们提出了一种数据驱动的深度强化学习框架，用于训练模拟角色的控制策略。我们证明该方法能够生成一系列具有挑战性的技能。生成的策略具有高度鲁棒性，在没有扰动的情况下能够产生自然运动，这些运动几乎与原始运动捕捉数据无法区分。我们的框架能够将技能重新定向到各种角色、环境和任务，多个策略可以组合成能够执行多种技能的复合策略。

尽管我们的实验说明了这种方法的灵活性，但在未来的工作中仍有大量局限性需要解决。首先，我们的策略需要一个与参考运动同步的阶段变量，该参考运动随时间线性前进。这限制了策略调整运动时序的能力，解除这一限制可以产生更自然和灵活的扰动恢复。我们的多片段集成方法对小数量片段效果良好，但尚未在大规模动作库上得到验证。用作角色低级伺服的PD控制器仍需要对每个个体的角色形态进行一些见解来正确设置。学习过程本身也非常耗时，通常每个技能需要几天时间，并且为每个策略独立执行。尽管我们在所有运动中都使用相同的模仿奖励，但这目前仍然基于手动定义的状态相似性度量。奖励项的权重也需要小心定义。

我们相信这项工作仍然为探索开辟了许多激动人心的方向。在未来的工作中，我们希望了解策略如何在机器人系统上部署，应用于运动、灵巧操作和其他任务。了解学习到的控制策略并将其与等效的人类策略进行比较将很有趣。我们希望集成多样化的技能，使角色能够执行更具挑战性的任务，并与环境进行更复杂的交互。结合层次结构很可能有助于实现这一目标。

致谢

我们感谢康康·尹、李立斌和Ziva Dynamics为该项目提供参考运动数据，感谢匿名审稿人提出的宝贵意见，以及AWS提供的计算资源。这项研究得到了NSERC研究生奖学金、伯克利研究生奖学金和ONR PECASE N000141612723的资助。

参考文献

Shailen Agrawal、Shuo Shen和Michiel van de Panne。2013年。《基于物理的角色动画的多样化运动变化》。计算机动画研讨会(2013年)。

Shailen Agrawal and Michiel van de Panne. 2016. Task-based Locomotion. *ACM Trans. Graph.* 35, 4, Article 82 (July 2016), 11 pages.

Marc G. Bellemare, Sriram Srinivasan, Georg Ostrovski, Tom Schaul, David Saxton, and Rémi Munos. 2016. Unifying Count-Based Exploration and Intrinsic Motivation. *CoRR* abs/1606.01868 (2016). arXiv:1606.01868

Greg Brockman, Vicki Cheung, Ludwig Pettersson, Jonas Schneider, John Schulman, Jie Tang, and Wojciech Zaremba. 2016a. OpenAI Gym. *CoRR* abs/1606.01540 (2016). arXiv:1606.01540

Greg Brockman, Vicki Cheung, Ludwig Pettersson, Jonas Schneider, John Schulman, Jie Tang, and Wojciech Zaremba. 2016b. OpenAI Gym. arXiv:arXiv:1606.01540

Bullet. 2015. Bullet Physics Library. http://bulletphysics.org.

Stelian Coros, Philippe Beaudoin, and Michiel van de Panne. 2009. Robust Task-based Control Policies for Physics-based Characters. *ACM Trans. Graph. (Proc. SIGGRAPH Asia)* 28, 5 (2009), Article 170.

Stelian Coros, Philippe Beaudoin, and Michiel van de Panne. 2010. Generalized Biped Walking Control. *ACM Transctions on Graphics* 29, 4 (2010), Article 130.

M. Da Silva, Y. Abe, and J. Popovic. 2008. Simulation of Human Motion Data using Short-Horizon Model-Predictive Control. *Computer Graphics Forum* (2008).

Yan Duan, Xi Chen, Rein Houthooft, John Schulman, and Pieter Abbeel. 2016. Benchmarking Deep Reinforcement Learning for Continuous Control. *CoRR* abs/1604.06778 (2016). arXiv:1604.06778

Justin Fu, John Co-Reyes, and Sergey Levine. 2017. EX2: Exploration with Exemplar Models for Deep Reinforcement Learning. In *Advances in Neural Information Processing Systems 30*. Curran Associates, Inc., 2574–2584.

Sehoon Ha and C Karen Liu. 2014. Iterative training of dynamic skills inspired by human coaching techniques. *ACM Transactions on Graphics* 34, 1 (2014).

Perttu Hämäläinen, Joose Rajamäki, and C Karen Liu. 2015. Online control of simulated humanoids using particle belief propagation. *ACM Transactions on Graphics (TOG)* 34, 4 (2015), 81.

Nicolas Heess, Dhruva TB, Srinivasan Sriram, Jay Lemmon, Josh Merel, Greg Wayne, Yuval Tassa, Tom Erez, Ziyu Wang, S. M. Ali Eslami, Martin A. Riedmiller, and David Silver. 2017. Emergence of Locomotion Behaviours in Rich Environments. *CoRR* abs/1707.02286 (2017). arXiv:1707.02286

Nicolas Heess, Gregory Wayne, Yuval Tassa, Timothy P. Lillicrap, Martin A. Riedmiller, and David Silver. 2016. Learning and Transfer of Modulated Locomotor Controllers. *CoRR* abs/1610.05182 (2016). arXiv:1610.05182

Jonathan Ho and Stefano Ermon. 2016. Generative Adversarial Imitation Learning. In *Advances in Neural Information Processing Systems 29*. Curran Associates, Inc., 4565–4573.

Daniel Holden, Taku Komura, and Jun Saito. 2017. Phase-functioned Neural Networks for Character Control. *ACM Trans. Graph.* 36, 4, Article 42 (July 2017), 13 pages.

Daniel Holden, Jun Saito, and Taku Komura. 2016. A Deep Learning Framework for Character Motion Synthesis and Editing. *ACM Trans. Graph.* 35, 4, Article 138 (July 2016), 11 pages.

Rein Houthooft, Xi Chen, Yan Duan, John Schulman, Filip De Turck, and Pieter Abbeel. 2016. Curiosity-driven Exploration in Deep Reinforcement Learning via Bayesian Neural Networks. *CoRR* abs/1605.09674 (2016). arXiv:1605.09674

Yoonsang Lee, Sungeun Kim, and Jehee Lee. 2010a. Data-driven Biped Control. In *ACM SIGGRAPH 2010 Papers (SIGGRAPH '10)*. ACM, New York, NY, USA, Article 129, 8 pages.

Yoonsang Lee, Moon Seok Park, Taesoo Kwon, and Jehee Lee. 2014. Locomotion Control for Many-muscle Humanoids. *ACM Trans. Graph.* 33, 6, Article 218 (Nov. 2014), 11 pages.

Yongjoon Lee, Kevin Wampler, Gilbert Bernstein, Jovan Popović, and Zoran Popović. 2010b. Motion Fields for Interactive Character Locomotion. In *ACM SIGGRAPH Asia 2010 Papers (SIGGRAPH ASIA '10)*. ACM, New York, NY, USA, Article 138, 8 pages.

Sergey Levine, Jack M. Wang, Alexis Haraux, Zoran Popović, and Vladlen Koltun. 2012. Continuous Character Control with Low-Dimensional Embeddings. *ACM Transactions on Graphics* 31, 4 (2012), 28.

Libin Liu and Jessica Hodgins. 2017. Learning to Schedule Control Fragments for Physics-Based Characters Using Deep Q-Learning. *ACM Trans. Graph.* 36, 3, Article 29 (June 2017), 14 pages.

Libin Liu, Michiel van de Panne, and KangKang Yin. 2016. Guided Learning of Control Graphs for Physics-Based Characters. *ACM Transactions on Graphics* 35, 3 (2016).

Libin Liu, KangKang Yin, Michiel van de Panne, Tianjia Shao, and Weiwei Xu. 2010. Sampling-based Contact-rich Motion Control. *ACM Transctions on Graphics* 29, 4 (2010), Article 128.

Josh Merel, Yuval Tassa, Dhruva TB, Sriram Srinivasan, Jay Lemmon, Ziyu Wang, Greg Wayne, and Nicolas Heess. 2017. Learning human behaviors from motion capture by adversarial imitation. *CoRR* abs/1707.02201 (2017). arXiv:1707.02201

Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Andrei A. Rusu, Joel Veness, Marc G. Bellemare, Alex Graves, Martin Riedmiller, Andreas K. Fidjeland, Georg Ostrovski, Stig Petersen, Charles Beattie, Amir Sadik, Ioannis Antonoglou, Helen King, Dharshan Kumaran, Daan Wierstra, Shane Legg, and Demis Hassabis. 2015. Human-level control through deep reinforcement learning. *Nature* 518, 7540 (Feb. 2015), 529–533.

Igor Mordatch, Emanuel Todorov, and Zoran Popović. 2012. Discovery of Complex Behaviors Through Contact-invariant Optimization. *ACM Trans. Graph.* 31, 4, Article 43 (July 2012), 8 pages.

Uldarico Muico, Yongjoon Lee, Jovan Popović, and Zoran Popović. 2009. Contact-aware nonlinear control of dynamic characters. In *ACM Transactions on Graphics (TOG)*, Vol. 28. ACM, 81.

Ashvin Nair, Bob McGrew, Marcin Andrychowicz, Wojciech Zaremba, and Pieter Abbeel. 2017. Overcoming Exploration in Reinforcement Learning with Demonstrations. *CoRR* abs/1709.10089 (2017). arXiv:1709.10089

Xue Bin Peng, Glen Berseth, and Michiel van de Panne. 2015. Dynamic Terrain Traversal Skills Using Reinforcement Learning. *ACM Trans. Graph.* 34, 4, Article 80 (July 2015), 11 pages.

Xue Bin Peng, Glen Berseth, and Michiel van de Panne. 2016. Terrain-Adaptive Locomotion Skills Using Deep Reinforcement Learning. *ACM Transactions on Graphics (Proc. SIGGRAPH 2016)* 35, 4 (2016).

Xue Bin Peng, Glen Berseth, KangKang Yin, and Michiel van de Panne. 2017a. DeepLoco: Dynamic Locomotion Skills Using Hierarchical Deep Reinforcement Learning. *ACM Transactions on Graphics (Proc. SIGGRAPH 2017)* 36, 4 (2017).

Xue Bin Peng, Michiel van de Panne, and KangKang Yin. 2017b. Learning Locomotion Skills Using DeepRL: Does the Choice of Action Space Matter?. In *Proc. ACM SIGGRAPH / Eurographics Symposium on Computer Animation*.

Aravind Rajeswaran, Sarvjeet Ghotra, Sergey Levine, and Balaraman Ravindran. 2016. EPOpt: Learning Robust Neural Network Policies Using Model Ensembles. *CoRR* abs/1610.01283 (2016). arXiv:1610.01283

Aravind Rajeswaran, Vikash Kumar, Abhishek Gupta, John Schulman, Emanuel Todorov, and Sergey Levine. 2017. Learning Complex Dexterous Manipulation with Deep Reinforcement Learning and Demonstrations. *CoRR* abs/1709.10087 (2017). arXiv:1709.10087

Alla Safonova and Jessica K Hodgins. 2007. Construction and optimal search of interpolated motion graphs. In *ACM Transactions on Graphics (TOG)*, Vol. 26. ACM, 106.

John Schulman, Sergey Levine, Philipp Moritz, Michael I. Jordan, and Pieter Abbeel. 2015a. Trust Region Policy Optimization. *CoRR* abs/1502.05477 (2015). arXiv:1502.05477

John Schulman, Philipp Moritz, Sergey Levine, Michael I. Jordan, and Pieter Abbeel. 2015b. High-Dimensional Continuous Control Using Generalized Advantage Estimation. *CoRR* abs/1506.02438 (2015). arXiv:1506.02438

John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. 2017. Proximal Policy Optimization Algorithms. *CoRR* abs/1707.06347 (2017). arXiv:1707.06347

Dana Sharon and Michiel van de Panne. 2005. Synthesis of Controllers for Stylized Planar Bipedal Walking. In *Proc. of IEEE International Conference on Robotics and Animation*.

Kwang Won Sok, Manmyung Kim, and Jehee Lee. 2007. Simulating biped behaviors from human motion data. In *ACM Transactions on Graphics (TOG)*, Vol. 26. ACM, 107.

R. Sutton, D. Mcallester, S. Singh, and Y. Mansour. 2001. Policy Gradient Methods for Reinforcement Learning with Function Approximation. , 1057–1063 pages.

Richard S. Sutton and Andrew G. Barto. 1998. *Introduction to Reinforcement Learning* (1st ed.). MIT Press, Cambridge, MA, USA.

Jie Tan, Karen Liu, and Greg Turk. 2011. Stable Proportional-Derivative Controllers. *IEEE Comput. Graph. Appl.* 31, 4 (2011), 34–44.

Yuval Tassa, Tom Erez, and Emanuel Todorov. 2012. Synthesis and stabilization of complex behaviors through online trajectory optimization. In *Intelligent Robots and Systems (IROS), 2012 IEEE/RSJ International Conference on*. IEEE, 4906–4913.

Yee Whye Teh, Victor Bapst, Wojciech Marian Czarnecki, John Quan, James Kirkpatrick, Raia Hadsell, Nicolas Heess, and Razvan Pascanu. 2017. Distral: Robust Multitask Reinforcement Learning. *CoRR* abs/1707.04175 (2017). arXiv:1707.04175

Kevin Wampler, Zoran Popović, and Jovan Popović. 2014. Generalizing Locomotion Style to New Animals with Inverse Optimal Regression. *ACM Trans. Graph.* 33, 4, Article 49 (July 2014), 11 pages.

Jack M. Wang, Samuel R. Hamner, Scott L. Delp, Vladlen Koltun, and More Specifically. 2012. Optimizing locomotion controllers using biologically-based actuators and objectives. *ACM Trans. Graph* (2012).

Ronald J. Williams. 1992. Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Machine Learning* 8, 3 (01 May 1992), 229–256.

Jungdam Won, Jongho Park, Kwanyu Kim, and Jehee Lee. 2017. How to Train Your Dragon: Example-guided Control of Flapping Flight. *ACM Trans. Graph.* 36, 6, Article 198 (Nov. 2017), 13 pages.

Yuting Ye and C Karen Liu. 2010a. Optimal feedback control for character animation using an abstract model. In *ACM Transactions on Graphics (TOG)*, Vol. 29. ACM, 74.

Yuting Ye and C. Karen Liu. 2010b. Synthesis of Responsive Motion Using a Dynamic Model. *Computer Graphics Forum* 29, 2 (2010), 555–562.

KangKang Yin, Kevin Loken, and Michiel van de Panne. 2007. SIMBICON: Simple Biped Locomotion Control. *ACM Trans. Graph.* 26, 3 (2007), Article 105.

Shailen Agrawal和Michiel van de Panne。2016年。《基于任务的行走》。*ACM Trans. Graph.* 35, 4, 文章82 (2016年7月), 11页。Marc G. Bellemare、Sriram Srinivasan、Georg Ostrovski、Tom Schaul、David Saxton和Rémi Munos。2016年。《统一计数式探索和内在动机》。*CoRR* abs/1606.01868 (2016年)。arXiv:1606.01868 Greg Brockman、Vicki Cheung、Ludwig Pettersson、Jonas Schneider、John Schulman、Jie Tang和Wojciech Zaremba。2016a。《OpenAI Gym》。*CoRR* abs/1606.01540 (2016年)。arXiv:1606.01540 Greg Brockman、Vicki Cheung、Ludwig Pettersson、Jonas Schneider、John Schulman、Jie Tang和Wojciech Zaremba。2016b。《OpenAI Gym》。arXiv:arXiv:1606.01540 Bullet。2015年。《Bullet物理库》。http://bulletphysics.org。Stelian Coros、Philippe Beaudoin和Michiel van de Panne。2009年。《基于物理的角色鲁棒任务式控制策略》。*ACM Trans. Graph. (SIGGRAPH Asia*会议录*)*28, 5 (2009年), 文章170。Stelian Coros、Philippe Beaudoin和Michiel van de Panne。2010年。《通用双足行走控制》。*ACM Transactions on Graphics*29, 4 (2010年), 文章130。M. Da Silva、Y. Abe和J. Popovic。2008年。《使用短视界模型预测控制模拟人体运动数据》。*Computer Graphics Forum* (2008年)。Yan Duan、Xi Chen、Rein Houthooft、John Schulman和Pieter Abbeel。2016年。《连续控制深度强化学习的基准测试》。*CoRR* abs/1604.06778 (2016年)。arXiv:1604.06778 Justin Fu、John Co-Reyes和Sergey Levine。2017年。《EX2：用于深度强化学习的示例模型探索》。收录于神经信息处理系统进展第*30*卷。Curran Associates, Inc., 2574–2584。Sehoon Ha和C Karen Liu。2014年。《受人类指导技术启发的动态技能迭代训练》。*ACM Transactions on Graphics* 34, 1 (2014年)。Perttu Hämäläinen、Joose Rajamäki和C Karen Liu。2015年。《使用粒子信念传播模拟人形机器人在线控制》。*ACM Transactions on Graphics (TOG)* 34, 4 (2015年), 81。Nicolas Heess、Dhruva TB、Srinivasan Sriram、Jay Lemmon、Josh Merel、Greg Wayne、Yuval Tassa、Tom Erez、Ziyu Wang、S. M. Ali Eslami、Martin A. Riedmiller和David Silver。2017年。《丰富环境中运动行为的涌现》。*CoRR* abs/1707.02286 (2017年)。arXiv:1707.02286 Nicolas Heess、Gregory Wayne、Yuval Tassa、Timothy P. Lillicrap、Martin A. Riedmiller和David Silver。2016年。《调制运动控制器学习和迁移》。*CoRR* abs/1610.05182 (2016年)。arXiv:1610.05182 Jonathan Ho和Stefano Ermon。2016年。《生成对抗性模仿学习》。收录于神经信息处理系统进展第*29*卷。Curran Associates, Inc., 4565–4573。Daniel Holden、Taku Komura和Jun Saito。2017年。《用于角色控制的相位函数神经网络》。*ACM Trans. Graph.* 36, 4, 文章42 (2017年7月), 13页。Daniel Holden、Jun Saito和Taku Komura。2016年。《用于角色运动合成和编辑的深度学习框架》。*ACM Trans. Graph.* 35, 4, 文章138 (2016年7月), 11页。Rein Houthooft、Xi Chen、Yan Duan、John Schulman、Filip De Turck和Pieter Abbeel。2016年。《通过贝叶斯神经网络在深度强化学习中实现好奇心驱动探索》。*CoRR* abs/1605.09674 (2016年)。arXiv:1605.09674 Yoonsang Lee、Sungeun Kim和Jehee Lee。2010a。《数据驱动双足控制》。收录于*ACM SIGGRAPH 2010* 129, 8 ***SIGGRAPH '10***的*10*篇论文。ACM, New York, NY, USA, 文章页。Yoonsang Lee、Moon Seok Park、Taesoo Kwon和Jehee Lee。2014年。《多肌肉人形机器人的行走控制》。*ACM Trans. Graph.* 33, 6, 文章218 (2014年11月), 11页。Yongjoon Lee、Kevin Wampler、Gilbert Bernstein、Jovan Popović和Zoran Popović。2010b。《用于交互式角色行走的运动场》。收录于***ACM SIGGRAPH Asia 2010***论文。ACM, New York, NY, USA, 文章138, 8页。Sergey Levine、Jack M. Wang、Alexis Haraux、Zoran Popović和Vladlen Koltun。2012年。《使用低维嵌入的连续角色控制》。*ACM Transactions on Graphics* 31, 4 (2012年), 28。Libin Liu和Jessica Hodgins。2017年。《使用深度Q学习为基于物理的角色学习调度控制片段》。*ACM Trans. Graph.*36, 3, 文章29 (2017年6月), 14页。Libin Liu、Michiel van de Panne和KangKang Yin。2016年。《基于物理的角色控制图引导学习》。*ACM Transactions on Graphics* 35, 3 (2016年)。Libin Liu、KangKang Yin、Michiel van de Panne、Tianjia Shao和Weiwei Xu。2010年。《基于采样的接触丰富运动控制》。*ACM Transctions on Graphics* 29, 4 (2010年), 文章128。Josh Merel、Yuval Tassa、Dhruva TB、Sriram Srinivasan、Jay Lemmon、Ziyu Wang、Greg Wayne和Nicolas Heess。2017年。《通过对抗性模仿从运动捕捉中学习人类行为》。*CoRR* abs/1707.02201 (2017年)。arXiv:1707.02201 Volodymyr Mnih、Koray Kavukcuoglu、David Silver、Andrei A. Rusu、Joel Veness、Marc G. Bellemare、Alex Graves、Martin Riedmiller、Andreas K. Fidjeland、Georg Ostrovski、Stig Petersen、Charles Beattie、Amir Sadik、Ioannis Antonoglou、Helen King、Dharshan Kumaran、Daan Wierstra、Shane Legg和Demis Hassabis。2015年。《通过深度强化学习实现人类级控制》。*Nature* 518, 7540 (2015年2月), 529–533。

Igor Mordatch，Emanuel Todorov 和 Zoran Popović。2012年。《通过接触不变优化发现复杂行为》。*ACM Trans. Graph.* 31, 4, 文章43（2012年7月），8页。Uldarico Muico，Yongjoon Lee，Jovan Popović 和 Zoran Popović。2009年。《动态角色的接触感知非线性控制》。在 *ACM Transactions on Graphics (TOG)*，第28卷。ACM，81页。Ashvin Nair，Bob McGrew，Marcin Andrychowicz，Wojciech Zaremba 和 Pieter Abbeel。2017年。《使用演示克服强化学习中的探索》。*CoRR* abs/1709.10089（2017年）。arXiv:1709.10089 Xue Bin Peng，Glen Berseth 和 Michiel van de Panne。2015年。《使用强化学习的动态地形穿越技能》。*ACM Trans. Graph.* 34, 4, 文章80（2015年7月），11页。Xue Bin Peng，Glen Berseth 和 Michiel van de Panne。2016年。《使用深度强化学习的地形自适应运动技能》。*ACM Transactions on Graphics (Proc. SIGGRAPH 2016)* 35, 4（2016年）。Xue Bin Peng，Glen Berseth，KangKang Yin 和 Michiel van de Panne。2017a年。《DeepLoco：使用分层深度强化学习的动态运动技能》。*ACM Transactions on Graphics (Proc. SIGGRAPH 2017)* 36, 4（2017年）。Xue Bin Peng，Michiel van de Panne 和 KangKang Yin。2017b年。《使用深度RL学习运动技能：动作空间的选择是否重要？》。在 *Proc. ACM SIGGRAPH / Eurographics Symposium on Computer Animation*。Aravind Rajeswaran，Sarvjeet Ghotra，Sergey Levine 和 Balaraman Ravindran。2016年。《EPOpt：使用模型集成学习鲁棒神经网络策略》。*CoRR* abs/1610.01283（2016年）。arXiv:1610.01283 Aravind Rajeswaran，Vikash Kumar，Abhishek Gupta，John Schulman，Emanuel Todorov 和 Sergey Levine。2017年。《使用深度强化学习和演示学习复杂灵巧操作》。*CoRR* abs/1709.10087（2017年）。arXiv:1709.10087 Alla Safonova 和 Jessica K Hodgins。2007年。《插值运动图的构建和最优搜索》。在 *ACM TransactionsonGraphics (TOG)*，第26卷。ACM，106页。John Schulman，Sergey Levine，Philipp Moritz，Michael I. Jordan 和 Pieter Abbeel。2015a年。《信任域策略优化》。*CoRR* abs/1502.05477（2015年）。arXiv:1502.05477 John Schulman，Philipp Moritz，Sergey Levine，Michael I. Jordan 和 Pieter Abbeel。2015b年。《使用广义优势估计的高维连续控制》。*CoRR* abs/1506.02438（2015年）。arXiv:1506.02438 John Schulman，Filip Wolski，Prafulla Dhariwal，Alec Radford 和 Oleg Klimov。2017年。《近端策略优化算法》。*CoRR* abs/1707.06347（2017年）。arXiv:1707.06347 Dana Sharon 和 Michiel van de Panne。2005年。《用于风格化平面双足行走的控制器合成》。在 *Proc. of IEEE International Conferenceon Robotics and Animation*。Kwang Won Sok，Manmyung Kim 和 Jehee Lee。2007年。《从人体运动数据模拟双足行为》。在 *ACM Transactions on Graphics (TOG)*，第26卷。ACM，107页。R. Sutton，D. Mcallester，S. Singh 和 Y. Mansour。2001年。《用于函数逼近的强化学习的策略梯度方法》。，1057–1063页。Richard S. Sutton 和 Andrew G. Barto。1998年。《强化学习导论 (第1版)。MIT Press，马萨诸塞州剑桥，美国。Jie Tan，Karen Liu 和 Greg Turk。2011年。《稳定的比例-微分控制器》。*IEEE Comput. Graph. Appl.* 31, 4（2011年），34–44页。Yuval Tassa，Tom Erez 和 Emanuel Todorov。2012年。《通过在线轨迹优化合成和稳定复杂行为》。在 *Intelligent Robots and Systems (IROS), 2012 IEEE/RSJ International Conference on*。IEEE，4906–4913页。Yee Whye Teh，Victor Bapst，Wojciech Marian Czarnecki，John Quan，James Kirkpatrick，Raia Hadsell，Nicolas Heess 和 Razvan Pascanu。2017年。《Distral：鲁棒多任务强化学习》。*CoRR* abs/1707.04175（2017年）。arXiv:1707.04175 Kevin Wampler，Zoran Popović 和 Jovan Popović。2014年。《使用逆最优回归将运动风格推广到新动物》。*ACM Trans. Graph.* 33, 4, 文章49（2014年7月），11页。Jack M. Wang，Samuel R. Hamner，Scott L. Delp，Vladlen Koltun 和 More Specifically。2012年。《使用基于生物的执行器和目标优化运动控制器》。*ACM Trans. Graph* (2012年)。Ronald J. Williams。1992年。《用于连接主义强化学习的简单统计梯度跟随算法》。*Machine Learning* 8, 3（1992年5月1日），229–256页。Jungdam Won，Jongho Park，Kwanyu Kim 和 Jehee Lee。2017年。《如何训练你的龙：示例引导的扑翼飞行控制》。*ACM Trans. Graph.* 36, 6, 文章198（2017年11月），13页。Yuting Ye 和 C Karen Liu。2010a年。《使用抽象模型的角色动画最优反馈控制》。在 *ACM Transactions on Graphics (TOG)*，第29卷。ACM，74页。Yuting Ye 和 C. Karen Liu。2010b年。《使用动态模型合成响应性运动》。*Computer Graphics Forum* 29, 2（2010年），555–562页。KangKang Yin，Kevin Loken 和 Michiel van de Panne。2007年。《SIMBICON：简单双足运动控制》。*ACM Trans. Graph.* 26, 3（2007年），文章105。

SUPPLEMENTARY MATERIAL

A MULTI-STEP RETURNS

We will refer to the return $R_t = \sum_{l=0}^{T-t} \gamma^l r_{t+l}$ as the Monte-Carlo return. R_t provides an unbiased sample of the expected return at a given state, but due to stochasticity from the dynamics of the environment and policy, each reward r_t can be a random variable, the sum of which can result in a high variance estimator of the expected return. Alternatively, an n-step return can be used to provide a lower-variance estimator at the cost of introducing some bias. The n-step return can be computed by truncating the sum of returns after n steps, and approximating the return from the remaining steps using a value function $V(s)$:

$$R_t^{(n)} = \sum_{l=0}^{n-1} \gamma^l r_{t+l} + \gamma^n V(s_{t+n}).$$

$R_t^{(1)} = r_t + \gamma V(s_{t+1})$ results in the 1-step return commonly used in Q-learning [Mnih et al. 2015] and $R_t^{(\infty)} = \sum_{l=0}^{T-t} \gamma^t r_{t+l} = R_t$ recovers the original Monte-Carlo return, given that all rewards after the horizon T are 0. While $R_t^{(\infty)}$ provides an unbiased but high variance estimator, $R_t^{(1)}$ provides a biased but low variance estimator. Therefore, n acts as a trade-off between bias and variance for the value estimator.

Another method to trade off between the bias and variance of the estimator is to use a λ-return [Sutton and Barto 1998], calculated as an exponentially-weighted average of n-step returns with decay parameter λ:

$$R_t(\lambda) = (1-\lambda) \sum_{n=1}^{\infty} \lambda^{n-1} R_t^n.$$

Assuming all rewards after step T are 0, such that $R_t^n = R_t^{T-t}$ for all $n \geq T - t$, the infinite sum can be calculated according to

$$R_t(\lambda) = (1-\lambda) \sum_{n=1}^{T-t-1} \lambda^{n-1} R_t^n + \lambda^{T-t-1} R_t^{T-t}$$

$\lambda = 0$ reduces to the single-step return $R_t^{(1)}$, and $\lambda = 1$ recovers the Monte-Carlo return R_t. Intermediate values of $\lambda \in (0, 1)$ produces interpolants that can be used to balance the bias and variance of the value estimator.

Updating the value function using the temporal difference computed with the λ-return results in the TD(λ) algorithm [Sutton and Barto 1998]. Similarly, estimating the advantage with the λ-return, yields the generalized advantage estimator GAE(λ) [Schulman et al. 2015b]:

$$\hat{\mathcal{A}}_t = R_t(\lambda) - V(s_t).$$

Empirically, the λ-return has been shown to produce better performance than simply using an n-step return [Schulman et al. 2015b; Sutton and Barto 1998].

B OFF-POLICY LEARNING

In its previously stated form, the gradient of the expected return $\nabla_\theta J(\theta)$ is estimated with respect to the current policy parameters θ. As such, data collected from the current policy can be justified for use only in performing a single gradient step, after which a new batch of data is required to estimate the gradient with respect to the updated parameters. Data efficiency can be improved by introducing importance sampling, which provides an unbiased estimate of the policy gradient using only off-policy samples from an older policy $\pi_{\theta_{old}}$:

$$\nabla_\theta J(\theta) = \mathbb{E}_{s_t \sim d_{\theta_{old}}(s_t), a_t \sim \pi_{\theta_{old}}(a_t|s_t)} \left[w_t(\theta) \nabla_\theta \log(\pi_\theta(a_t|s_t)) \mathcal{A}_t \right]$$

$$w_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{old}}(a_t|s_t)}$$

The importance-sampled policy gradient can be interpreted as optimizing the surrogate objective

$$L^{IS}(\theta) = \mathbb{E}_{s_t \sim d_{\theta_{old}}(s_t), a_t \sim \pi_{\theta_{old}}(a_t|s_t)} \left[w_t(\theta) \mathcal{A}_t \right].$$

With L^{IS}, the same batch of data can be used to perform multiple update steps for θ. While in theory importance sampling allows for the policy gradient to be estimated with data collected from any policy with sufficient support, we will consider only the case where the data is collected from a previous set of policy parameters.

C PROXIMAL POLICY OPTIMIZATION

In practice, the policy gradient estimator discussed thus far suffers from high variance, often leading to instability during learning, where the policy's performance fluctuates drastically between iterations. While problems due to noisy gradients can be mitigated by using large batches of data per update, policy gradient algorithms can still be extremely unstable. Trust region methods have been proposed as a technique for improving the stability of policy gradient algorithms [Schulman et al. 2015a]. Trust Region Policy Optimization (TRPO) optimizes the same objective $J(\theta)$ but includes an additional KL-divergence constraint to prevent the behaviour of the current policy π_θ from deviating too far from the previous policy $\pi_{\theta_{old}}$.

$$\begin{aligned} \max_\theta \quad & J(\theta) \\ \text{s.t.} \quad & \mathbb{E}_{s_t \sim d_\theta(s_t)} \left[KL\left(\pi_{\theta_{old}}(\cdot|s_t) | \pi_\theta(\cdot|s_t) \right) \right] \leq \delta_{KL} \end{aligned}$$

δ_{KL} is a hyper parameters that defines the trust region. TRPO has been successfully applied to solve a wide variety of challenging RL problems [Duan et al. 2016; Ho and Ermon 2016; Rajeswaran et al. 2016]. However, ensuring that the constraint is satisfied can be difficult, and it is often instead enforced approximately using the conjugate gradient algorithm with an adaptive stepsize selected by a line search to ensure the approximate constraint is satisfied [Schulman et al. 2015a].

Proximal Policy Optimization (PPO) is a variant of TRPO, where the hard constraint is replaced by optimizing a surrogate loss [Schulman et al. 2017]. In this work, we will be concerned mainly with the clipped surrogate loss $L^{CLIP}(\theta)$ defined according to

$$L^{CLIP}(\theta) = \mathbb{E}_{s_t, a_t} \left[\min\left(w_t(\theta) \mathcal{A}_t, \text{clip}\left(w_t(\theta), 1-\epsilon, 1+\epsilon \right) \mathcal{A}_t \right) \right].$$

When $\theta = \theta_{old}$, $w_t(\theta) = 1$, but as θ is changed, the likelihood ratio

补充材料

多步回报

我们将返回 $R_t = \sum_{T-t l=0} {}^{Y} l r_{t+l}$称为蒙特卡洛回报。 R_t提供了在给定状态下预期回报的无偏样本，但由于环境的动态性和策略的随机性，每个奖励r_t 可能是一个随机变量，其总和可能导致预期回报的高方差估计器。或者，可以使用n步回报来提供较低方差估计器，但代价是引入一些偏差。n步回报可以通过截断n步后的回报总和，并使用值函数 $V(s)$ 来近似剩余步骤的回报来计算：

$$R_t^{(n)} = \sum_{l=0}^{n-1} \gamma^l r_{t+l} + \gamma^n V(s_{t+n}).$$

R(1) t = rt + γV (st+1) 产生 Q-learning 中常用的 1 步回报 [Mnih 等人 2015]，而 R(∞) t = ∑T −tl=0 Ytrt+l = Rt 在视野 T 之后的所有回报为 0 的情况下，会恢复原始的蒙特卡洛回报。虽然 R(∞) t 提供一个无偏但方差高的估计器，R(1) t 提供一个有偏但方差低的估计器。因此，n 在值估计器中充当偏差和方差之间的权衡。

另一种权衡估计器偏差和方差的方法是使用一个 λ-回报 [Sutton 和 Barto 1998]，计算为具有衰减参数 λ 的 n 步回报的指数加权平均值：n:

$$R_t(\lambda) = (1-\lambda) \sum_{n=1}^{\infty} \lambda^{n-1} R_t^n.$$

假设步骤 T后的所有奖励都为 0，即 $R_{nt} = R_{T-tt}$对所有 $n \geq T - t$ 成立，则无限和可以根据

$$R_t(\lambda) = (1-\lambda) \sum_{n=1}^{T-t-1} \lambda^{n-1} R_t^n + \lambda^{T-t-1} R_t^{T-t}$$

$\lambda = 0$ 简化为单步回报 $R_{(1)\,t}$，并且 $\lambda = 1$ 恢复蒙特卡洛回报 R_t。中间值 $\lambda \in (0, 1)$ 产生的插值可以用来平衡值估计器的偏差和方差。

使用与 λ-回报计算的时序差分来更新值函数，会得到 TD(λ) 算法 [Sutton and Barto 1998]。类似地，使用 λ-回报估计优势，会得到广义优势估计器 GAE(λ) [Schulman et al. 2015b]:

$$\hat{\mathcal{A}}_t = R_t(\lambda) - V(s_t).$$

经验上， λ-回报已被证明比简单地使用 n-步回报 [Schulman et al. 2015b; Sutton and Barto 1998] 产生更好的性能。

B 离线策略学习

在其先前所述的形式中，预期回报$\nabla_\theta J(\theta)$的梯度是针对当前策略参数 θ 进行估计的。因此，从当前策略收集的数据仅可用于执行单个梯度步，之后需要一个新的

一批数据是必要的，用以估计关于更新参数的梯度。通过引入重要性采样，可以提高数据效率，它使用来自旧策略的离线策略样本$\pi_{\theta_{old}}$，仅提供策略梯度的无偏估计。

$$\nabla_\theta J(\theta) = \mathbb{E}_{s_t \sim d_{\theta_{old}}(s_t), a_t \sim \pi_{\theta_{old}}(a_t|s_t)} \left[w_t(\theta) \nabla_\theta \log(\pi_\theta(a_t|s_t)) \mathcal{A}_t \right]$$

$$w_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{old}}(a_t|s_t)}$$

重要性采样的策略梯度可以解释为优化代理目标

$$L^{IS}(\theta) = \mathbb{E}_{s_t \sim d_{\theta_{old}}(s_t), a_t \sim \pi_{\theta_{old}}(a_t|s_t)} \left[w_t(\theta) \mathcal{A}_t \right].$$

使用 L^{IS} 时，相同的数据批次可用于对 θ 进行多次更新步骤。虽然理论上重要性采样允许使用来自任何具有足够支持策略收集的数据来估计策略梯度，但我们将仅考虑数据来自先前策略参数集的情况。

C 近端策略优化

在实践中，之前讨论的策略梯度估计器存在高方差问题，这通常导致学习过程中的不稳定，其中策略的性能在迭代之间剧烈波动。虽然可以通过使用每个更新的大批量数据来缓解由于噪声梯度引起的问题，但策略梯度算法仍然可能非常不稳定。信任域方法已被提出作为一种提高策略梯度算法稳定性 [Schulman et al. 2015a] 的技术。信任域策略优化（TRPO）优化相同的目标 $J(\theta)$，但包含一个额外的 KL 散度约束，以防止当前策略 π_θ 的行为偏离先前策略 $\pi_{\theta_{old}}$ 太远。

$$\begin{aligned} \max_\theta \quad & J(\theta) \\ \text{s.t.} \quad & \mathbb{E}_{s_t \sim d_\theta(s_t)} \left[KL\left(\pi_{\theta_{old}}(\cdot|s_t) | \pi_\theta(\cdot|s_t) \right) \right] \leq \delta_{KL} \end{aligned}$$

δ_{KL} 是一个定义信任域的超参数。TRPO已被成功应用于解决各种具有挑战性的强化学习问题 [Duan等人，2016年；Ho和Ermon，2016年；Rajeswaran等人 2016]。然而，确保约束得到满足可能很困难，通常使用共轭梯度算法近似执行约束，并通过线搜索选择自适应步长来确保近似约束得到满足[Schulman等人，2015a]。

近端策略优化（PPO）是TRPO的一种变体，其中硬约束被替换为优化代理损失 [Schul- man等人 2017]。在这项工作中，我们将主要关注根据θ定义的裁剪代理损失L^{CLIP}。

$$L^{CLIP}(\theta) = \mathbb{E}_{s_t, a_t} \left[\min\left(w_t(\theta) \mathcal{A}_t, \text{clip}\left(w_t(\theta), 1-\epsilon, 1+\epsilon \right) \mathcal{A}_t \right) \right].$$

当 $\theta = \theta_{旧的}$， $w_t(\theta) = 1$，但随着 θ 的改变，似然比

moves away from 1. Therefore, the likelihood ratio can be interpreted as a measure of the similarity between two policies. To discourage the policies from deviating too far apart, clip $(w_t(\theta), 1-\epsilon, 1+\epsilon)$ sets the gradient to zero whenever the ratio is more than ϵ away from 1. This term therefore serves a similar function to the KL-divergence constraint in TRPO. The minimum is then taken between the clipped and unclipped advantages to create a lower bound of $L(\theta)$.

D LEARNING ALGORITHM

Algorithm 1 summarizes the common learning procedure used to train all policies. Policy updates are performed after a batch of $m = 4096$ samples has been collected. Minibatches of size $n = 256$ are then sampled from the data for each gradient step. A discount factor $\gamma = 0.95$ is used for all motions. $\lambda = 0.95$ is used for both TD(λ) and GAE(λ). The likelihood ratio clipping threshold is set to $\epsilon = 0.2$. A stepsize of $\alpha_v = 10^{-2}$ is used for the value function. A policy step size of $\alpha_\pi = 5 \times 10^{-5}$ is used for the humanoid and Atlas, and $\alpha_\pi = 2 \times 10^{-5}$ for the dragon and T-Rex. Once gradients have been computed, the network parameters are updated using stochastic gradient descent with momentum 0.9. The same hyperparameter settings are used for all characters and skills, with the exception of the step size. Humanoid policies for imitating individual skills typically require about 60 million samples to train, requiring about 2 days on an 8-core machine. All simulation and network updates are performed on the CPU and no GPU acceleration is used.

ALGORITHM 1: Proximal Policy Optimization

$\theta \leftarrow$ random weights
$\psi \leftarrow$ random weights
while not done **do**
  $s_0 \leftarrow$ sample initial state from reference motion
  Initialize character to state s_0
  for step = 1, …, m **do**
    $s \leftarrow$ start state
    $a \sim \pi_\theta(a|s)$
    Apply a and simulate forward one step
    $s' \leftarrow$ end state
    $r \leftarrow$ reward
    record (s, a, r, s') into memory D
  end for

  $\theta_{old} \leftarrow \theta$
  for each update step **do**
    Sample minibatch of n samples $\{(s_i, a_i, r_i, s_i')\}$ from D

    Update value function:
    for each (s_i, a_i, r_i, s_i') **do**
      $y_i \leftarrow$ compute target values using TD(λ)
    end for
    $\psi \leftarrow \psi + \alpha_v \left(\frac{1}{n} \sum_i \nabla_\psi V_\psi(s_i)(y_i - V(s_i))\right)$

    Update policy:
    for each (s_i, a_i, r_i, s_i') **do**
      $\mathcal{A}_i \leftarrow$ compute advantage using V_ψ and GAE
      $w_i(\theta) \leftarrow \frac{\pi_\theta(a_i|s_i)}{\pi_{\theta_{old}}(a_i|s_i)}$
    end for
    $\theta \leftarrow$
    $\theta + \alpha_\pi \frac{1}{n} \sum_i \nabla_\theta \min\left(w_i(\theta)\mathcal{A}_i, \text{clip}\left(w_i(\theta), 1-\epsilon, 1+\epsilon\right)\mathcal{A}_i\right)$
  end for
end while

偏离 1。因此，似然比可以被解释为衡量两个策略之间相似性的指标。为了防止策略偏离太远，片段 $(w_t(\theta),\ 1-\epsilon, 1+\epsilon)$ 在比率偏离 1 超过 ϵ 时将梯度设为零。因此，该术语在 TRPO 中与 KL 散度约束具有类似的功能。然后，在裁剪和未裁剪的优势值之间取最小值，以创建 $L(\theta)$ 的下界。

深度学习算法

算法1总结了用于训练所有策略的通用学习过程。策略更新是在收集了一批<样式id='1'>m</样式> = 4096 样本之后执行的。然后从数据中为每个梯度步骤采样大小为<样式id='4'>n</样式> = 256 的mini-batch。对所有运动使用折扣因子 $\gamma = 0.95$。$\lambda = 0.95$用于TD(λ)和GAE(λ)。似然比裁剪阈值设置为 $\epsilon = 0.2$。值函数使用步长α<样式id='13'>v</样式>= 10^{-2}。人形机器人和Atlas使用 $\alpha_\pi = 5 \times 10^{-5}$ 的策略步长，龙和三角龙使用 $\alpha_\pi = 2 \times 10^{-5}$。计算梯度后，使用动量为0.9的随机梯度下降来更新网络参数。所有角色和技能都使用相同的超参数设置，步长除外。模仿单个技能的人形策略通常需要大约6000万样本进行训练，在8核机器上需要大约2天。所有模拟和网络更新都在CPU上执行，未使用GPU加速。

算法 1： 近端策略优化

1: $\theta \leftarrow$ 随机权重
2: $\psi \leftarrow$ 随机权重
3: **当** 未完成 **时**
4: $s_0 \leftarrow$ 从参考运动中采样初始状态
5: 将角色初始化为状态s_0
6: **对于** 步骤 = 1, …, m **执行**
7: $s \leftarrow$ 初始状态
8: $a \sim \pi_\theta(a|s)$
9: 应用 a 并向前模拟一步
10: $s' \leftarrow$ 终止状态
11: $r \leftarrow$奖励
12: 将 (s, a, r, s') 记录到记忆库 D
13: **end for**

14: $\theta_{old} \leftarrow \theta$
15: **for**each 更新步骤 **do**
16: 从 D 中采样 n 个样本 $\{(s_i, a_i, r_i, s_i')\}$

17: 更新值函数：
18: **对于** 每一 (s_i, a_i, r_i, s_i') **都**
19: $y_i \leftarrow$ 使用TD(λ)计算目标值
20: **结束for循环**
$\psi \leftarrow \psi + \alpha_v \left(\frac{1}{n} \sum_i \nabla_\psi V_\psi(s_i)(y_i - V(s_i))\right)$
21:

22: 更新策略：
23: **for** 每个 (s_i, a_i, r_i, s_i') **执行**
24: $A_i \leftarrow$ 使用V_ψ和GAE计算优势
25: $w_i(\theta) \leftarrow \frac{\pi_\theta(a_i|s_i)}{\pi_{\theta_{旧}(人工智能)}}$
26: **结束for循环**
27: $\theta \leftarrow \theta + \alpha_\pi \frac{1}{n} \sum_i \nabla_\theta \min\left(w_i(\theta)A_i, 片段\left(w_i(\theta),\ 1-\epsilon, 1+\epsilon\right)A_i\right)$
28: **结束for循环**
29: **结束while**

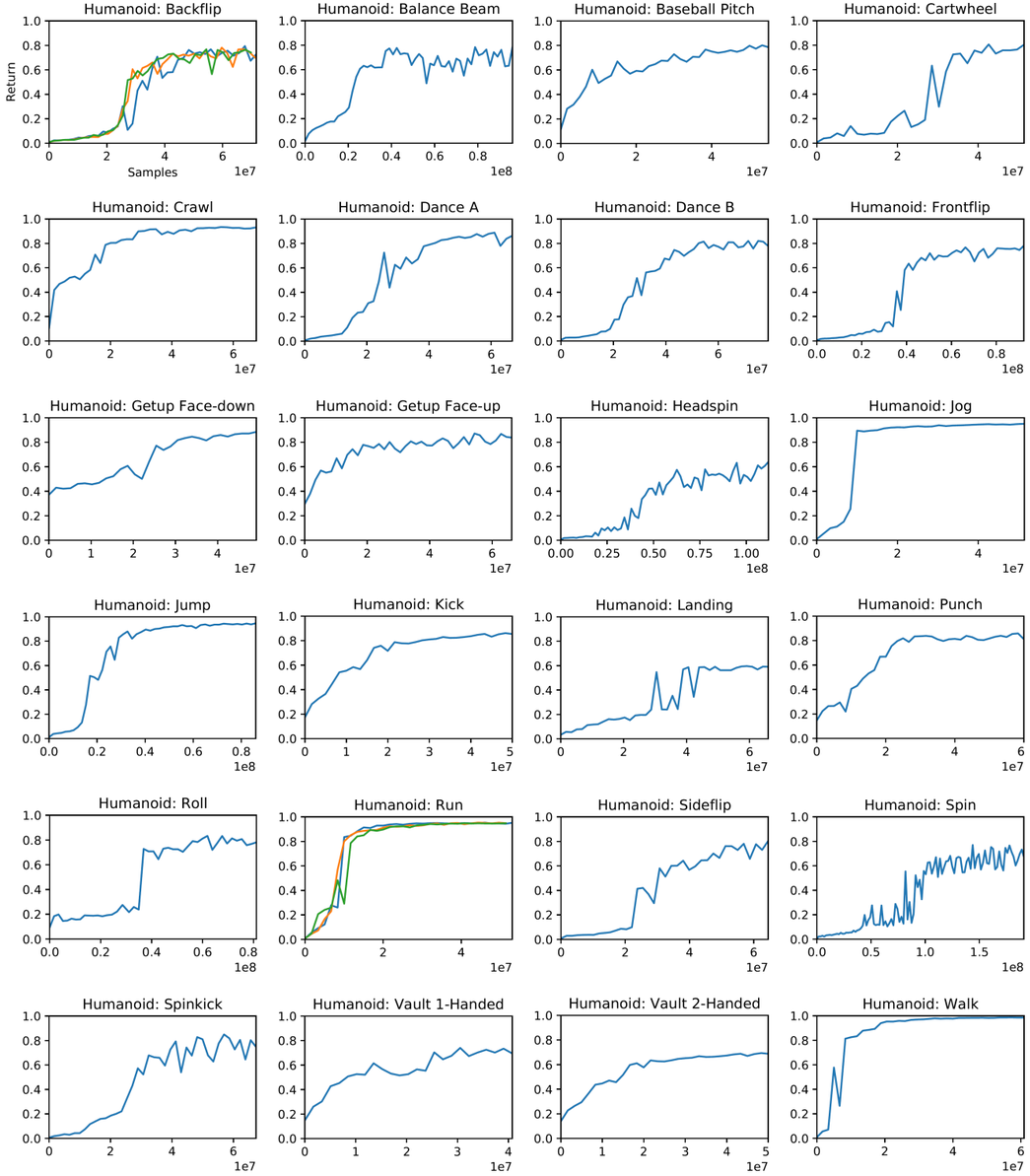


Fig. 12. Learning curves of the humanoid imitating individual motion clips. Performance is calculated as the mean return over 32 episodes. The returns are normalized by the minimum and maximum possible return per episode. Due to the time needed to train each policy, performance statistics are collected only from one run of the training process for the majority of skills. For the backflip and run policies, we collected statistics from 3 training runs using different random seeds. Performance appears consistent across multiple runs, and we have observed similar behaviours for many of the other skills.

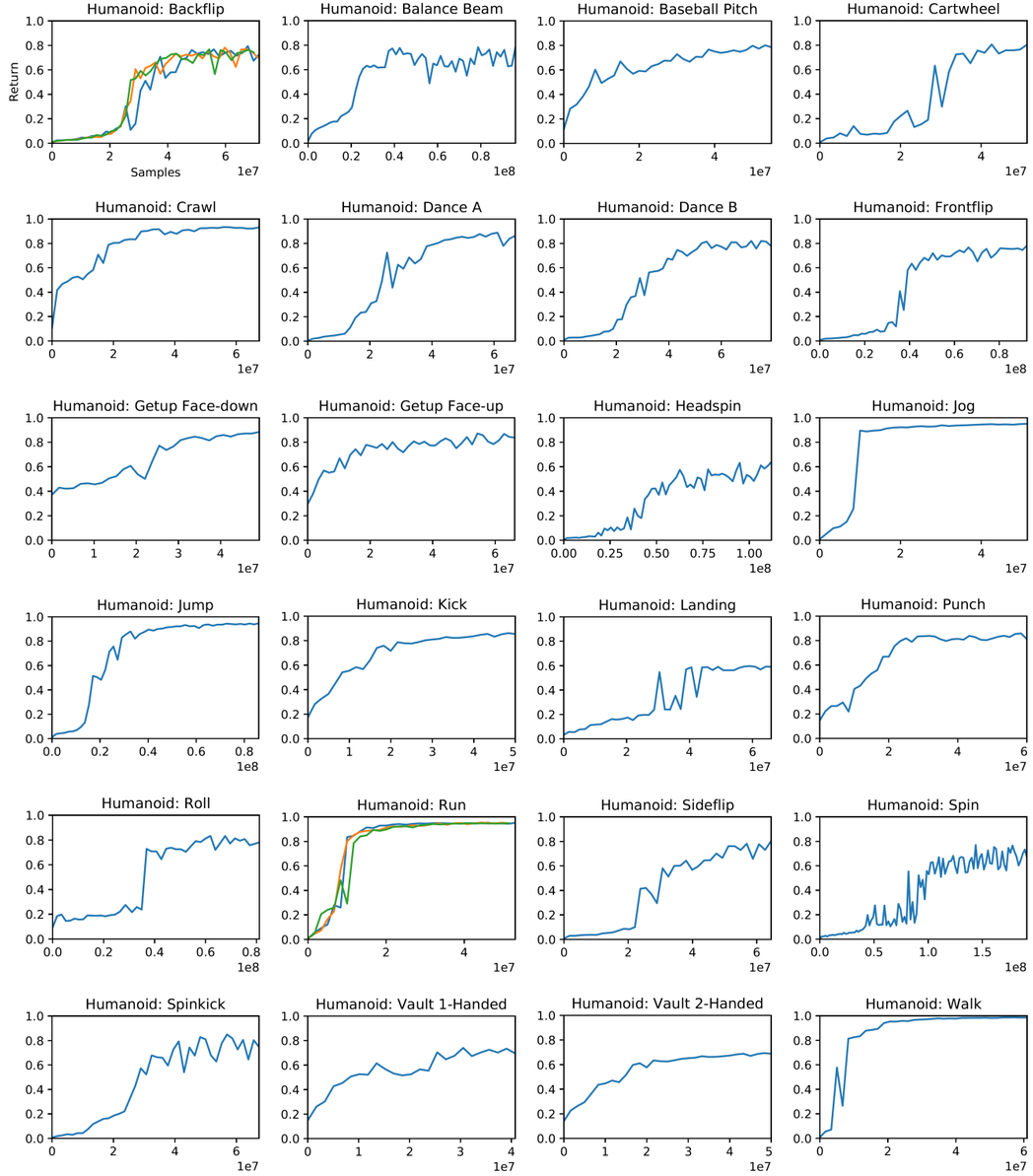


图12。人形机器人模仿单个动作片段的学习曲线。性能计算为32个回合的平均回报。回报值通过每个回合的最小和最大可能回报进行归一化。由于训练每个策略需要的时间，大多数技能的性能统计数据仅从训练过程的一个运行中收集。对于后空翻和跑步策略，我们使用不同的随机种子收集了3次训练运行的数据。性能在多次运行中表现一致，并且我们观察到许多其他技能也表现出类似的行为。

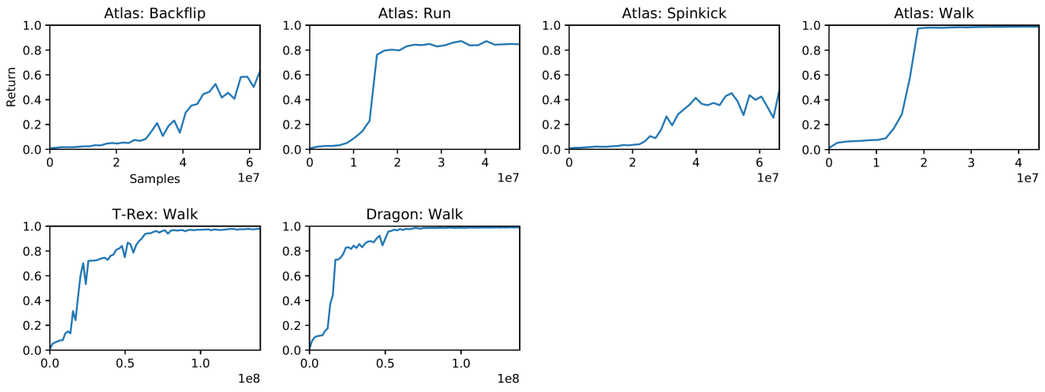


Fig. 13. Learning curves of the Atlas, T-Rex, and dragon imitating individual motion clips.

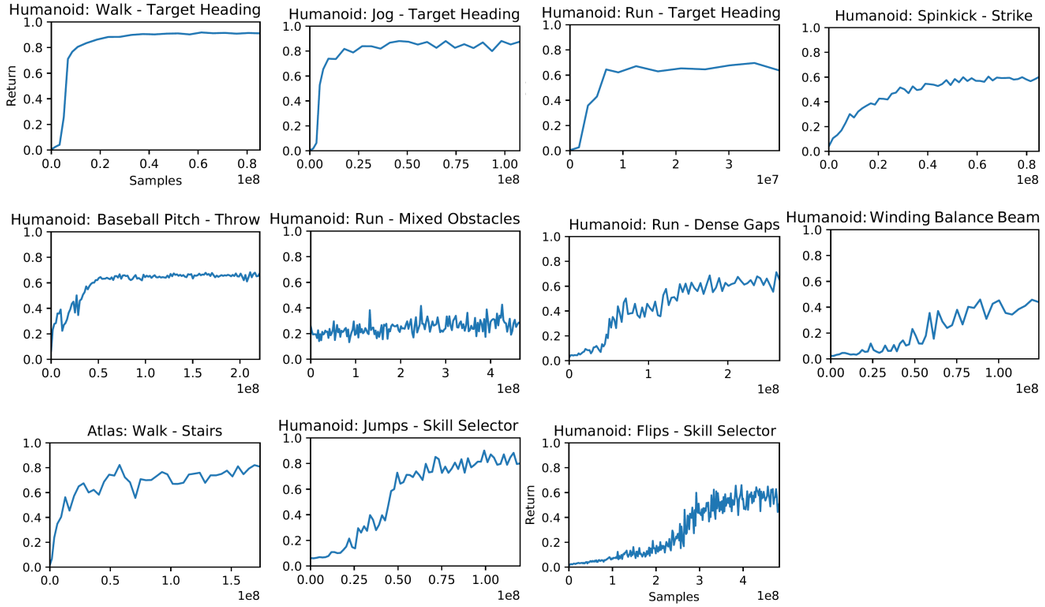


Fig. 14. Learning curves of policies trained to imitate motion clips while fulfilling additional task objectives.

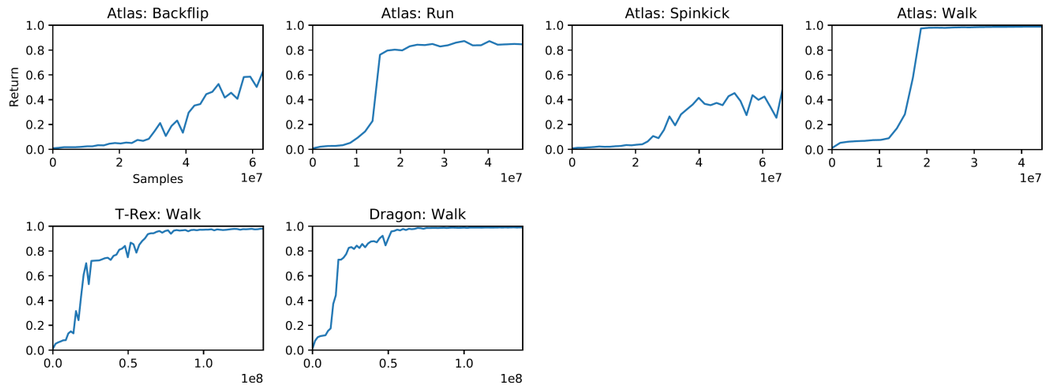


图13。Atlas、三角龙和龙模仿单个片段的学习曲线。

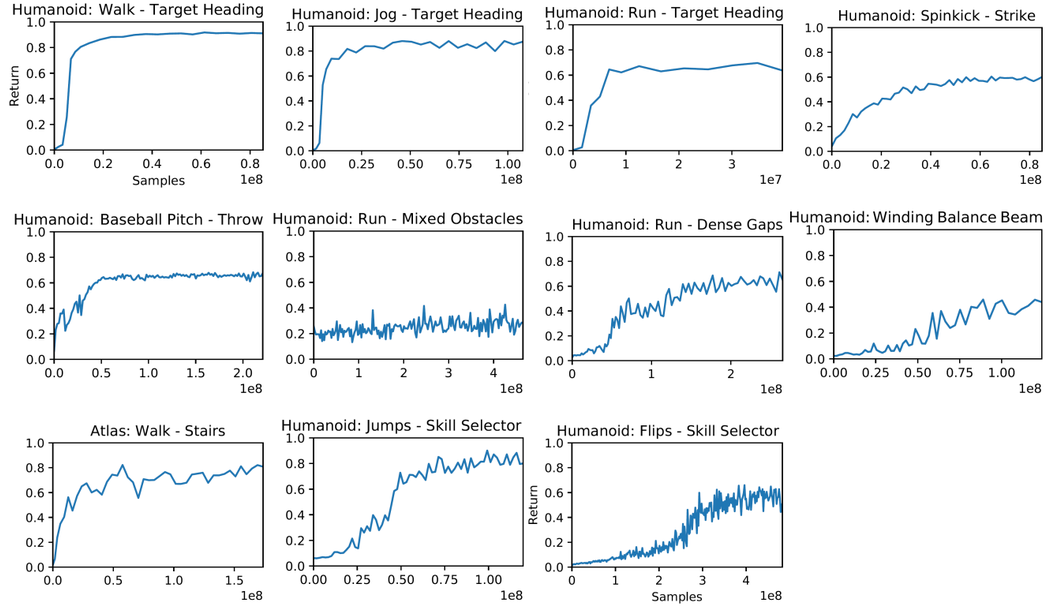


图14。训练策略模仿运动片段同时满足附加任务目标的学习曲线。